

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

Opening Minds • Shaping the Future
啟迪思維 • 成就未來

Not to be redistributed to Course Hero or any other public websites

DSAI6707 Advanced Computational Intelligence

Sem 1 2026/2027

Searching Techniques

What is searching?

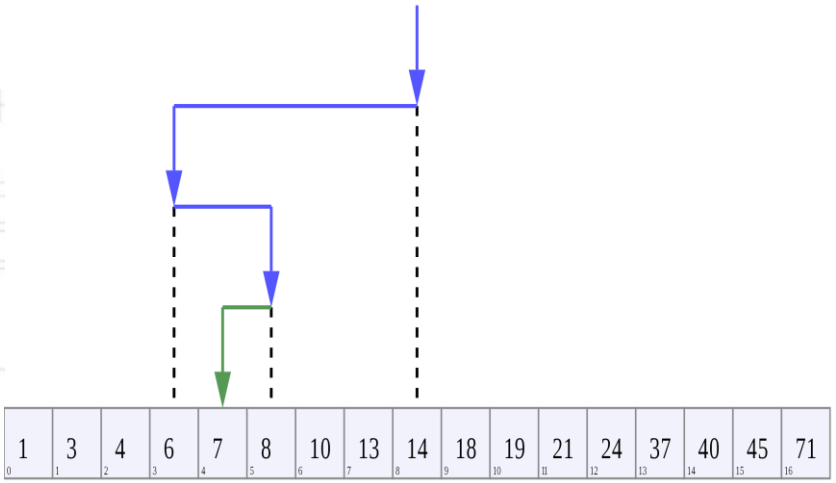
- Searching is the process of looking for the solution of a problem through a set of states.
- Search conditions include
  - Current state: the step of where we are
  - Goal state: the step where the solution is found
  - Actions: move one state to another
  - Cost: how many steps needed to find the solution.
- Solution: A path from the current state to the goal state.

What searching matters?

- An agent can find a sequence of actions that achieves its goals when no single action will do.

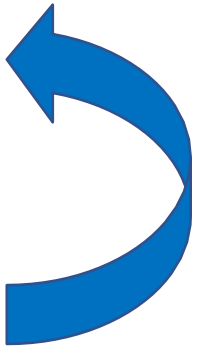


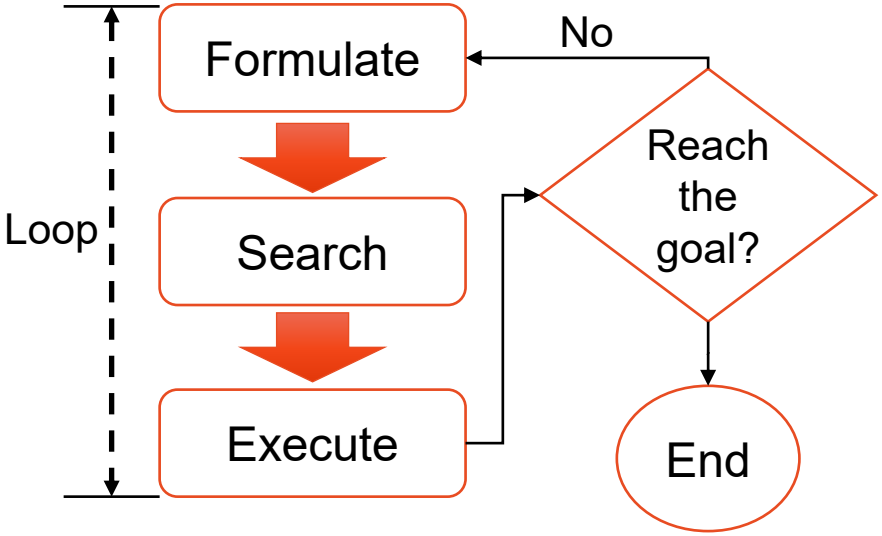
Linear Search algorithm



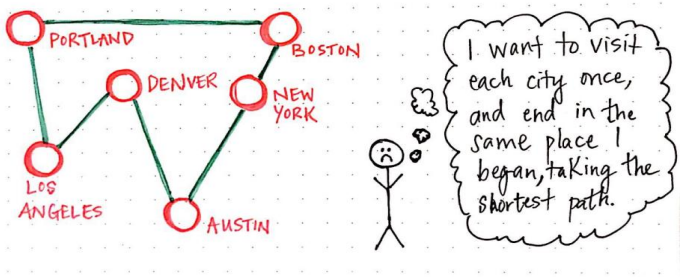
Binary Search algorithm

- Almost every AI problem depends on a search procedure to find the solution. Searching proceeds as follows:
  - Check the current state (reach the goal?);
  - Execute allowable actions to move to the next state;
  - Check if the new state is the goal state.
    - Yes. Done!
    - No. The new state becomes the current state.



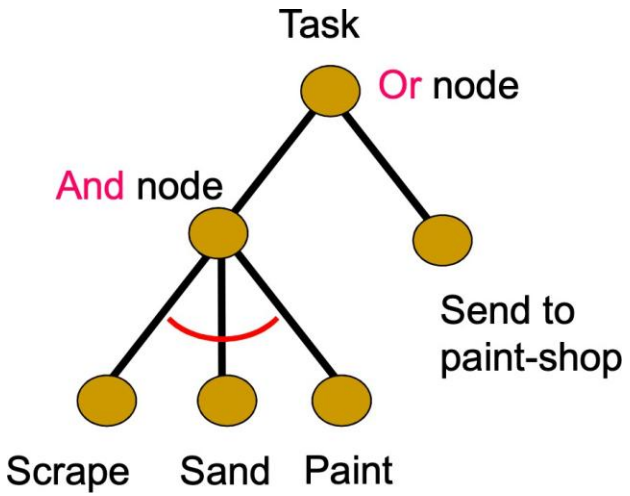


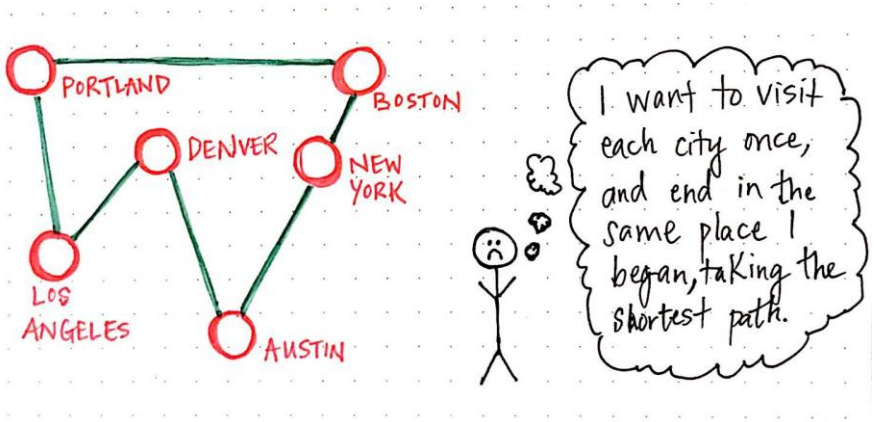
The process is repeated until a solution is found or the state space is exhausted.



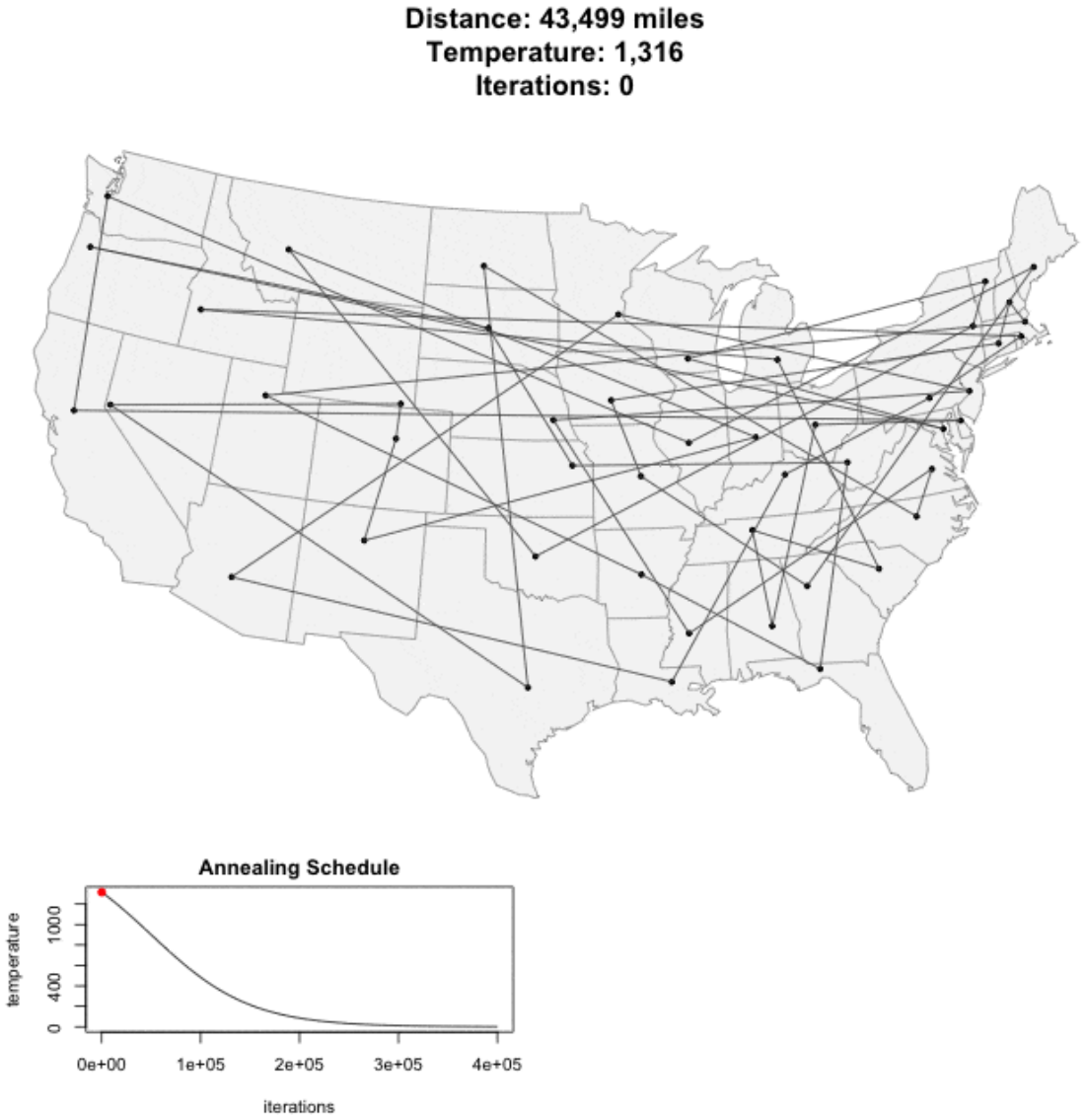
Graph and Tree Representation

- Nodes: the states in a searching process
- Edges: transitions between states (actions).
- And-Or Graph:
  - 'And' node: a node with all edges emanating from it connected by a curved line.
  - 'Or' node: a node has no line connecting its edges to indicate that any emanating path can be a solution.





Traveling Salesman Problem (TSP)

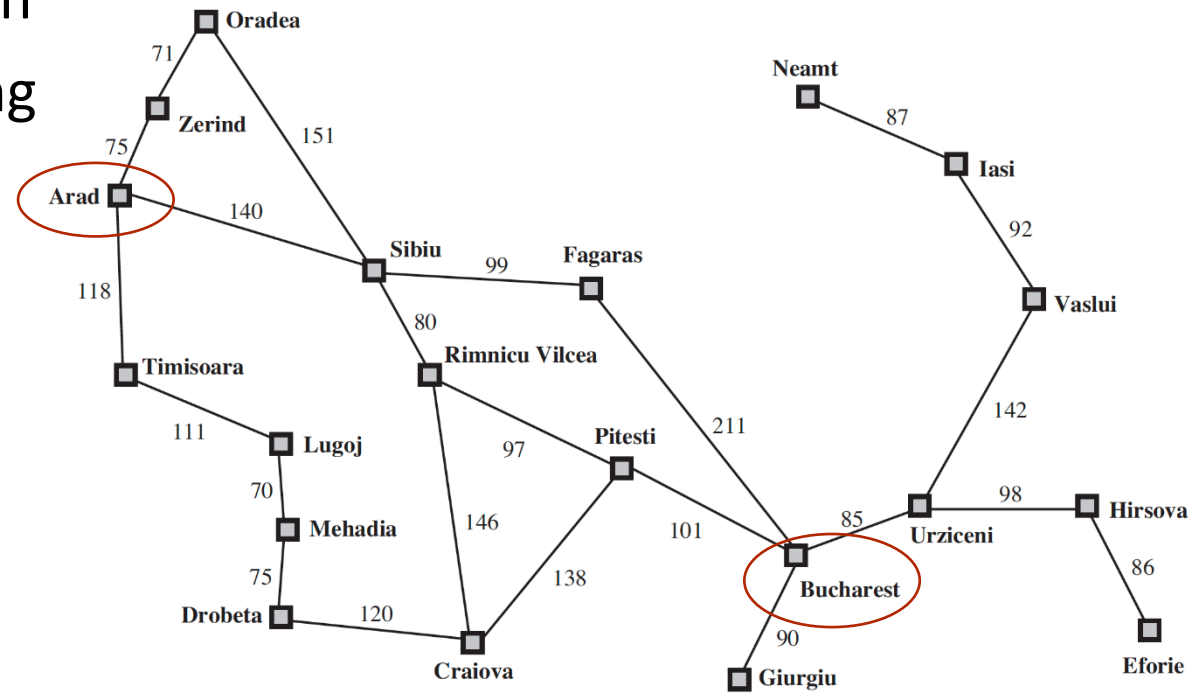

Distance: 43,499 miles
Temperature: 1,316
Iterations: 0
Annealing Schedule
temperature
0
400
1000
0e+00
1e+05
2e+05
3e+05
4e+05
iterations

Informed (Heuristic) Search

- AI problem solver employs heuristics in two situations:
  - A problem may not have an exact solution because of ambiguities in problem statement or available data. Example: medical diagnosis.
  - A problem may have an exact solution, but the computational cost of finding it may be prohibitive.
- Uninformed (blind) searches are normally very inefficient. Information about the problem can sometimes be used to guild the search more efficiently.
- Informed search explores the node that is most “likely” to be the nearest to a goal state.
- Informed search works successfully in many cases, but its success is not guaranteed.

Informed (Heuristic) Search

- Hill Climbing
- A* Algorithm Search
- Simulated Annealing
- Genetic Algorithm
- .....



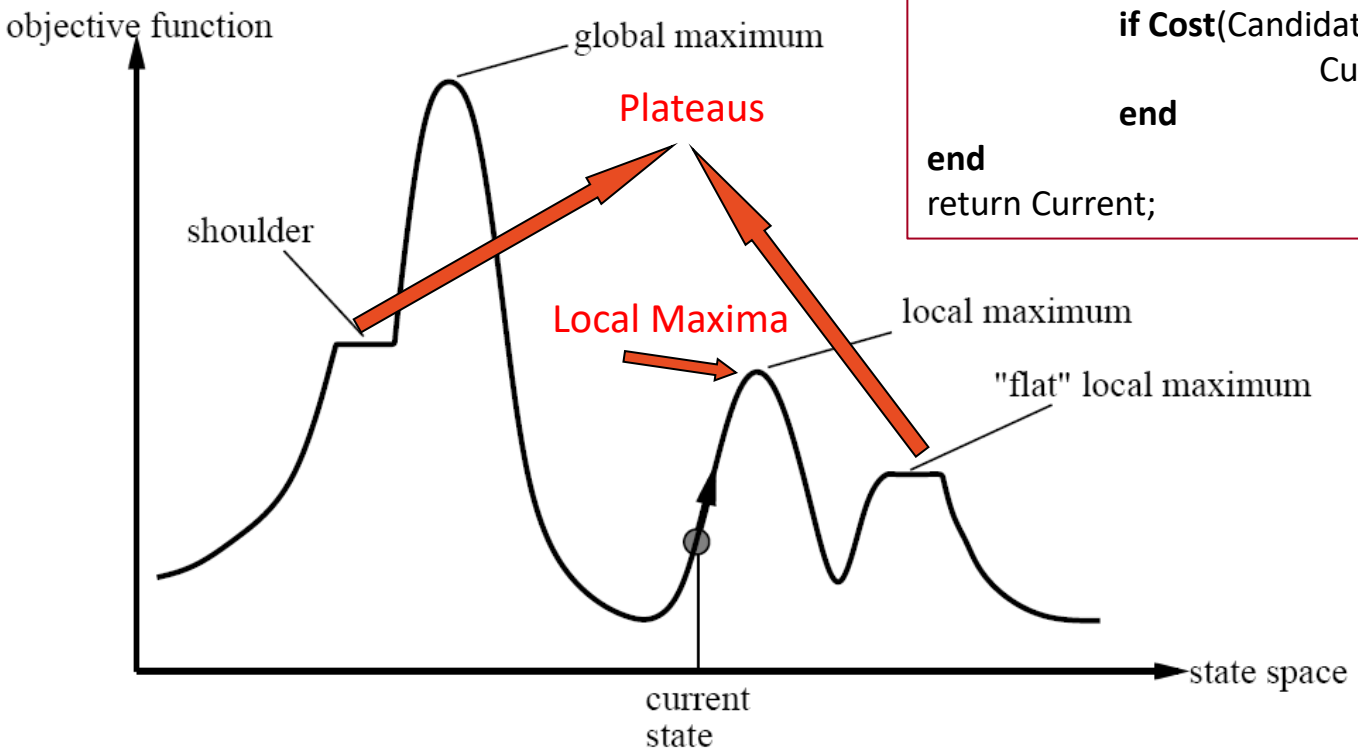
A simplified road map of part of Romania

Hill Climbing

- Consider an eager but blind mountain climber: go uphill along the steepest possible path until can go no further up.
- Hill-climbing expands the current state and evaluates its children.
  - The **best child** is selected for further expansion; neither its siblings nor its parent are retained.
- Because it keeps no history, the algorithm cannot recover from failures of its strategy.
- Hill climbing performs well if the distance estimate (quality measurement) is locally consistent. That is, the node with the shortest distance estimate will eventually lead to the goal.
- Hill climbing may encounter the problem of stuck/trap at local optima.

Problems with Hill Climbing

Input: $Iter_{max}$, ProblemSize (Minimization)
Output: Current
Current ← **RandomSolution**(ProblemSize);
for each $iter_i \in Iter_{max}$ **do**
    Candidate ← **RandomNeighbour**(Current);
    if Cost(Candidate) ≤ **Cost**(Current) **then**
        Current ← Candidate;
    end
end
return Current;



The A* Search

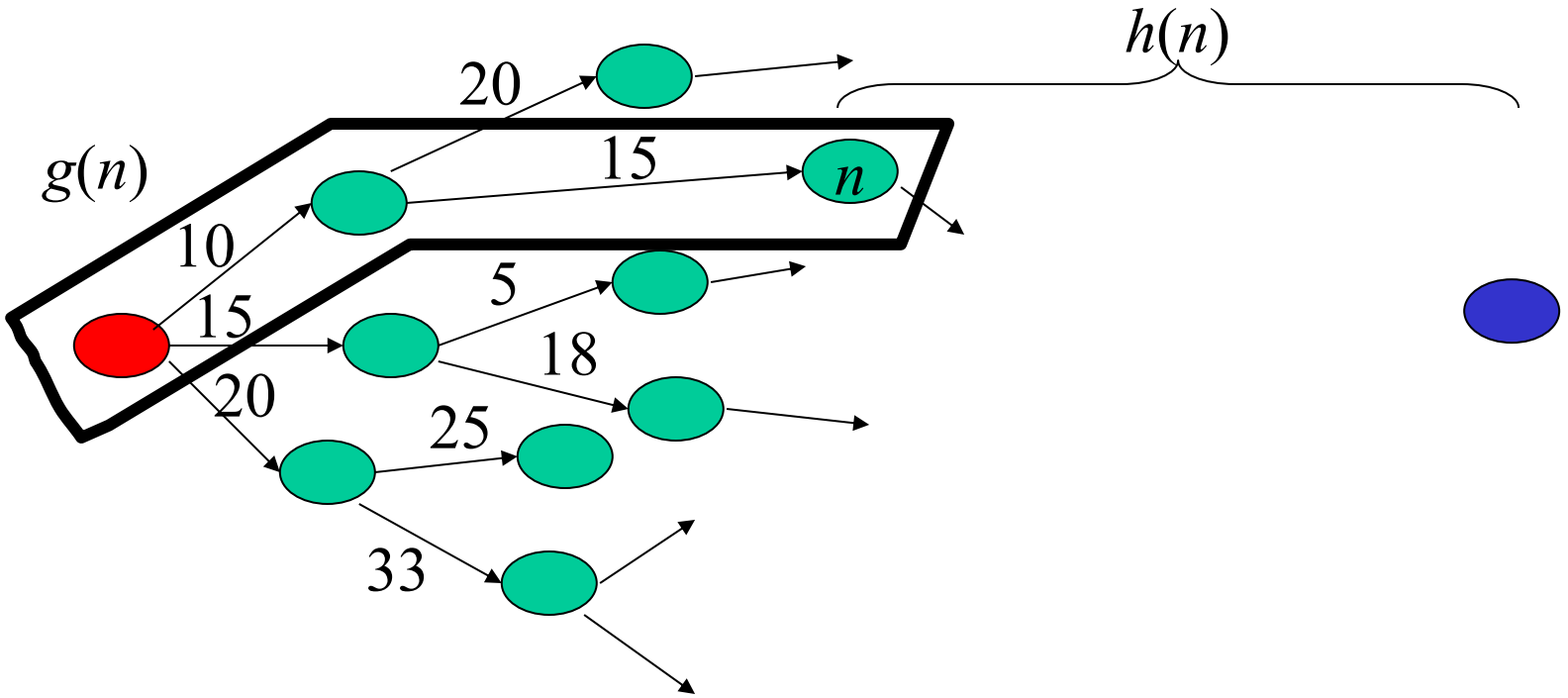
- A* is an algorithm that uses heuristic to guide search while ensuring that it will compute a path with minimum cost.

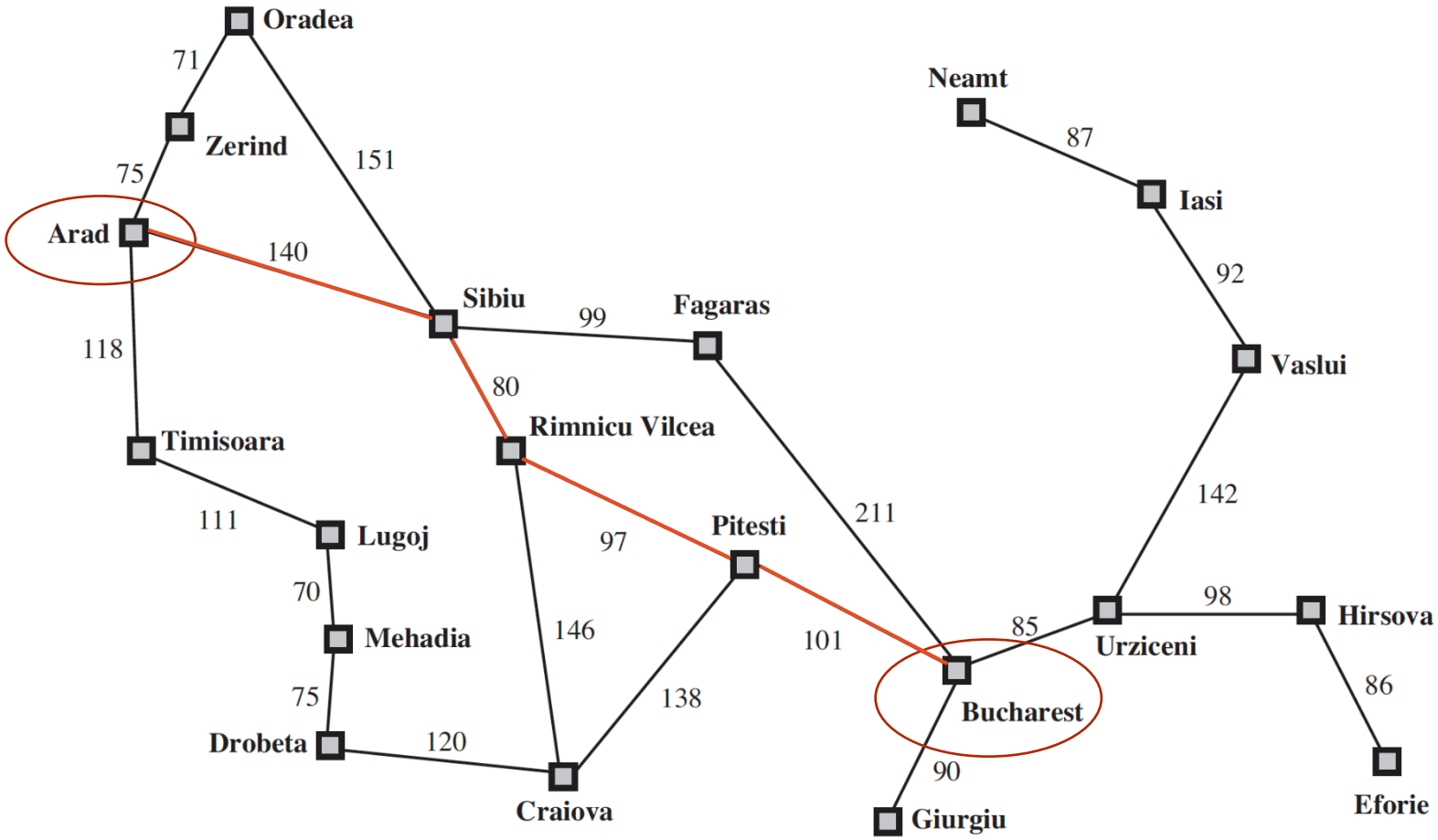
- A* computes the function $f(n) = g(n) + h(n)$
  - $h(n)$: "estimated cost"
  - $g(n)$: "actual cost"

  - The $h(n)$ value guides search toward heuristically promising states while the $g(n)$ value prevents search from persisting indefinitely on a fruitless search.
  - The function $h(n)$ relies upon some heuristic information available from the problem domain. It changes from heuristic to heuristic, and from problem to problem.
  - The design of good heuristics is a difficult, empirical problem. $h(n)$ in general weights how close node n is to the goal.

A* Search

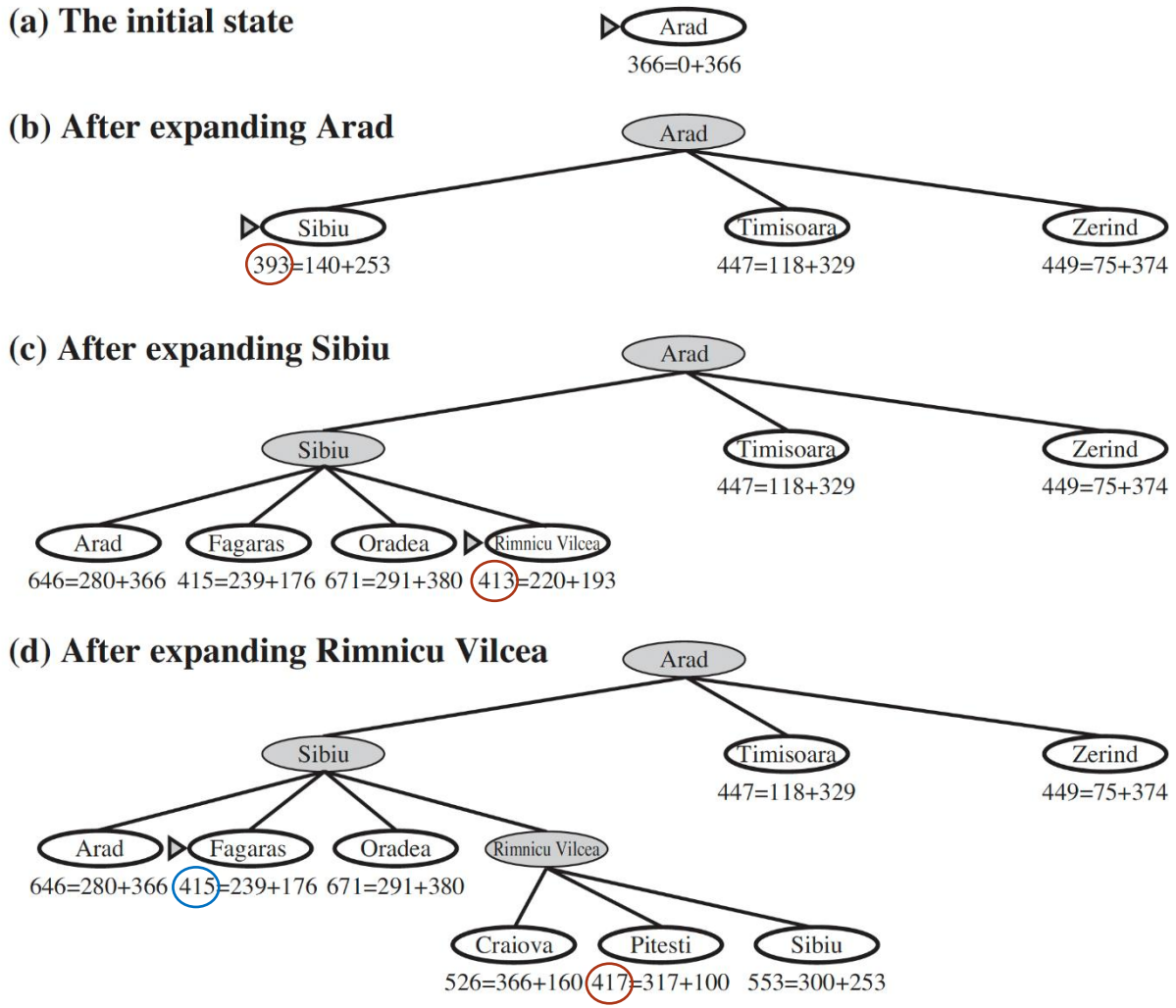
- A*: minimize $f(n) = g(n) + h(n)$
  - $g(n)$ = "cost from the starting node to reach n"
  - $h(n)$ = "estimate of the cost of the cheapest path from n to the goal node"





A simplified road map of part of Romania

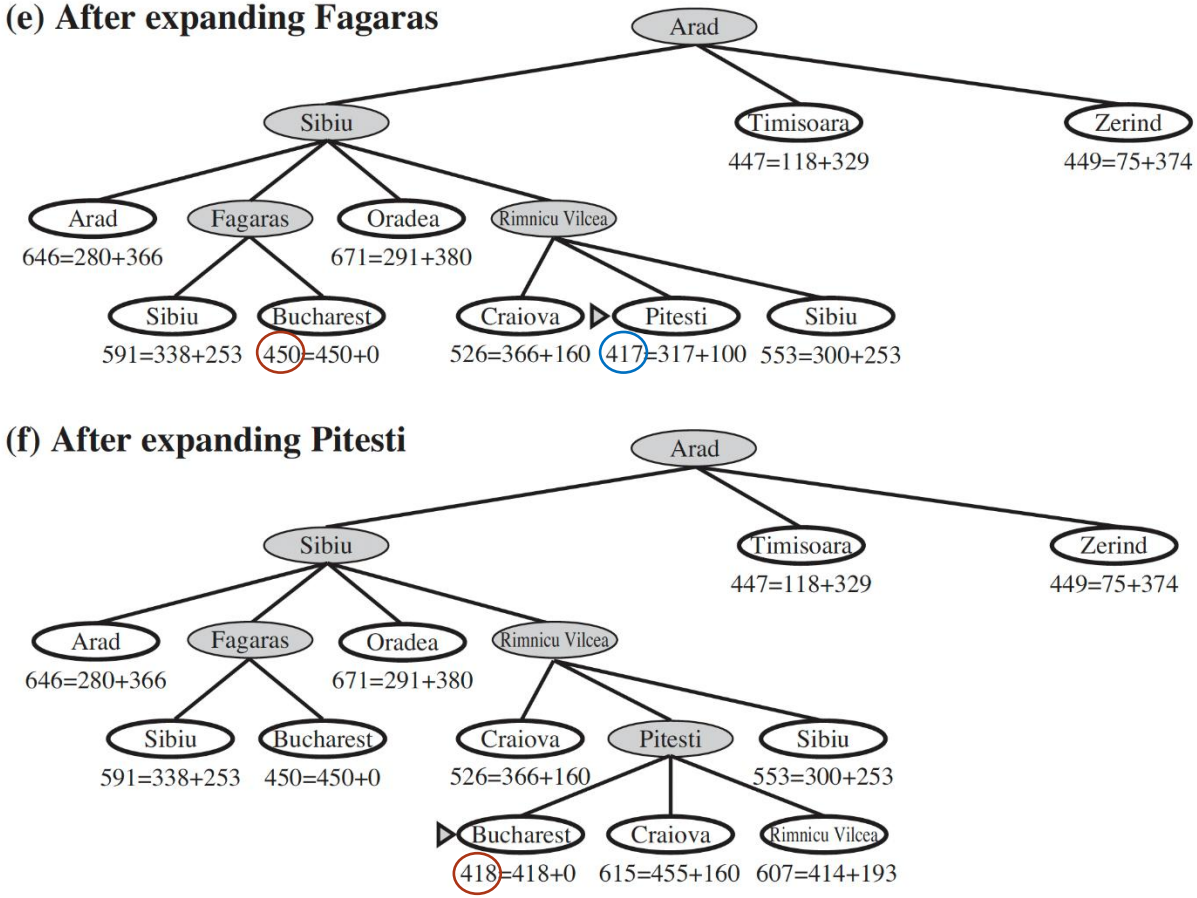
Stages in an A∗ search for Bucharest. Nodes are labeled with $f = g + h$.



Values of h_{SLD}: straight-line distances to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

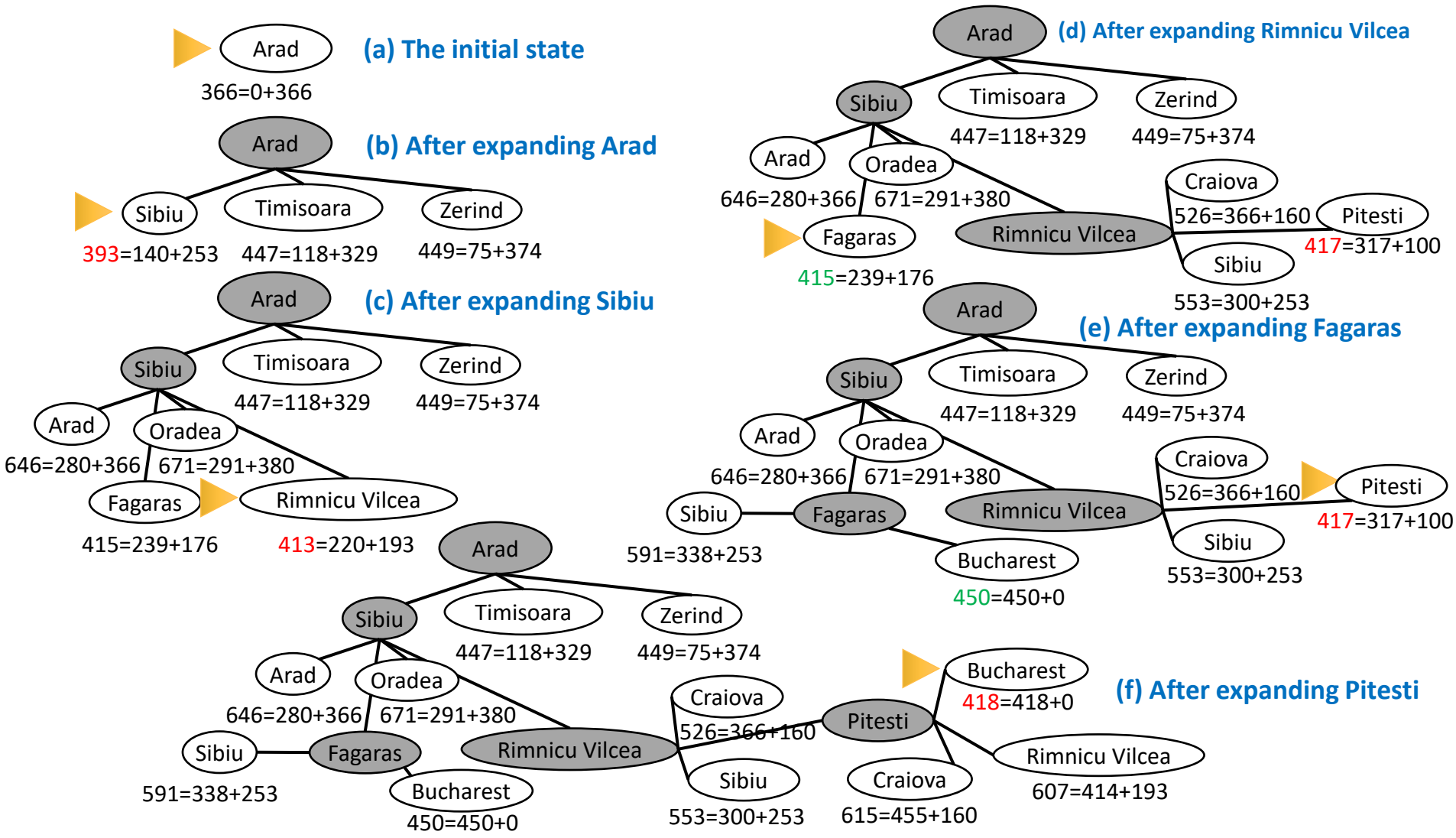
Stages in an A∗ search for Bucharest. Nodes are labeled with $f = g + h$.



Values of h_{SLD}: straight-line distances to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

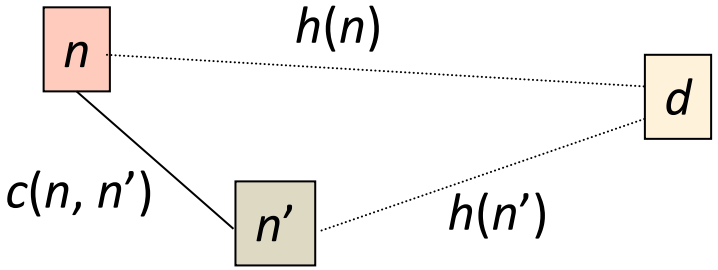
Stages in an A* Search for Bucharest



Properties of A*

- A* generates an optimal solution if $h(n)$ is **admissible** (never overestimates the cost to reach the destination node, i.e., the cost it estimates to reach the goal is not higher than the lowest possible cost from the current point in the path).
- A* generates an optimal solution if $h(n)$ is **consistent**, i.e., for every node n and for every successor node n' of n:

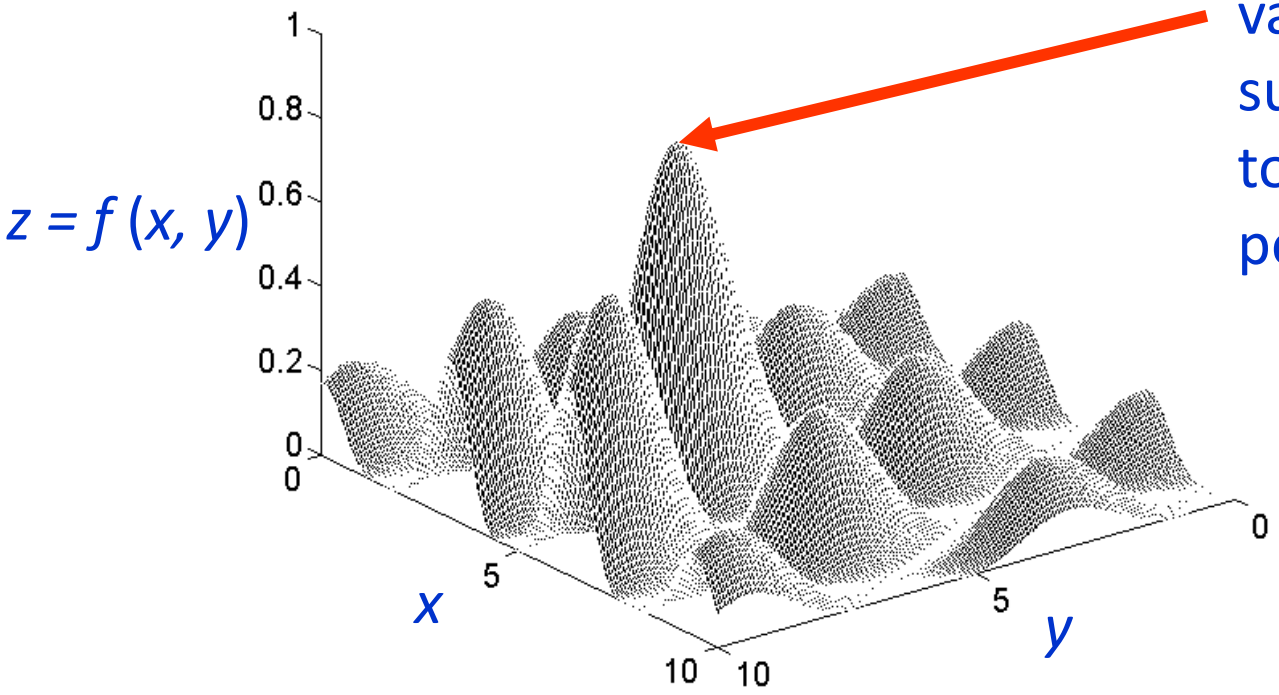
$$h(n) \leq c(n, n') + h(n')$$



- If $h(n)$ is consistent, f along any path is non-decreasing.
- If $h(n)$ is consistent, then $h(n)$ is admissible.

Optimization

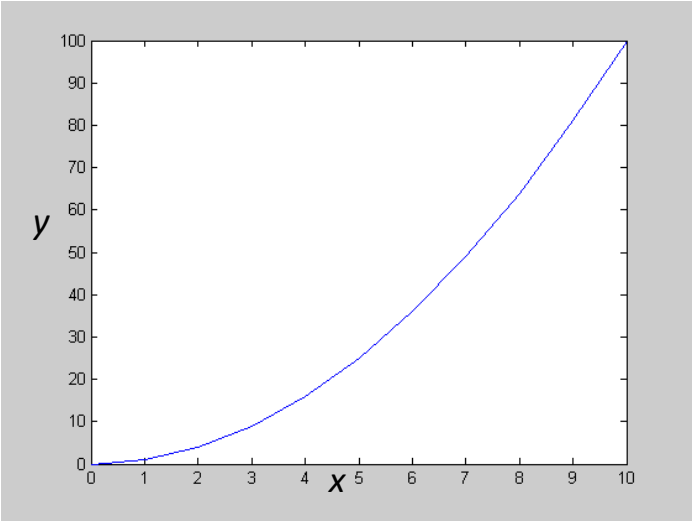
- What is an optimization problem?



What are the values of *x* and *y* such that *z* is equal to the value at the peak?

maximization?
minimization?

An Optimization Problem



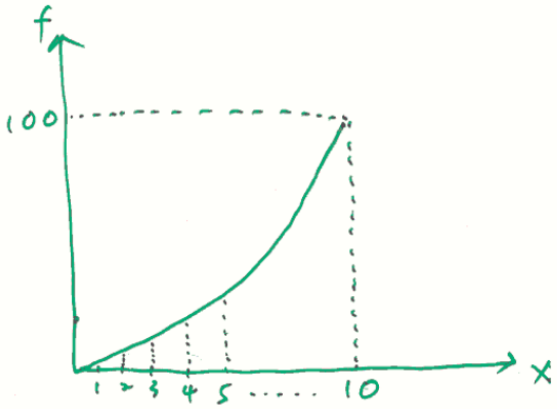
$$y = f(x) = x^2 \quad where \quad x \in [0, 10]$$

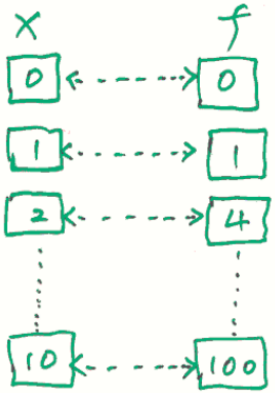
Find the value of *x* that gives the minimum of *y*

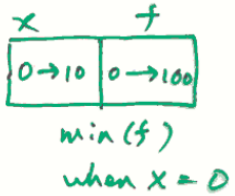
$$\frac{df}{dx} = 0 = 2x \Rightarrow x = 0$$

min (f) when $x = 0$

Alternatively,



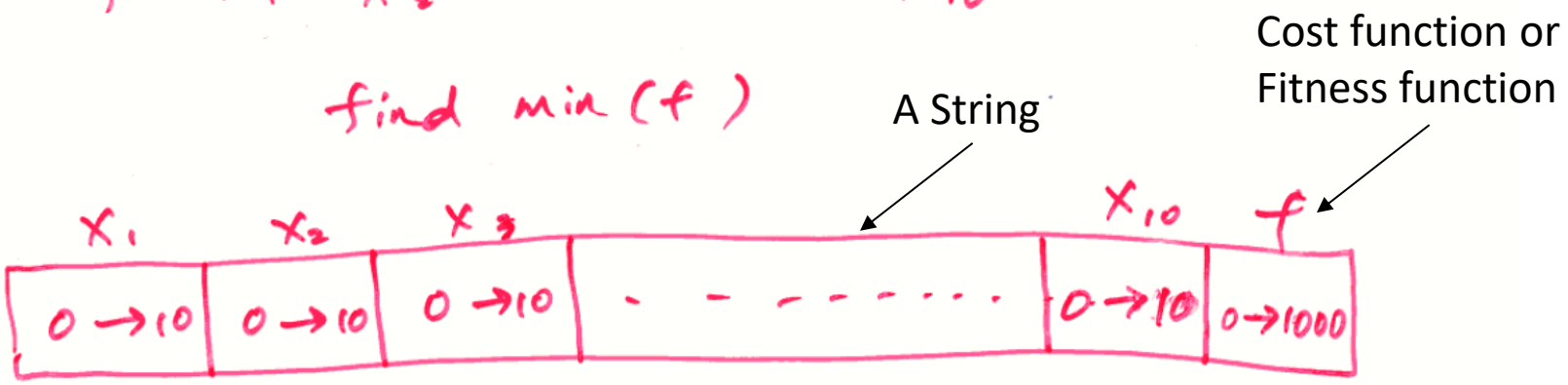




Consider if now we have 10 parameters $X_1, X_2, \ldots X_{10}$

$f = X_1^2 + X_2^2 + \ldots\ldots\ldots X_{10}^2$

find min (f)



Consider if now we have 10 parameters $X_1, X_2, \ldots X_{10}$

$f = X_1^2 + X_2^2 + \ldots\ldots\ldots X_{10}^2$

find min (f)

Cost function or Fitness function

A String



time needed ?

$11^{10} \times 0.01 \text{ sec} = 3002 \text{ days}$

$= 8.22 \text{ years}$

11^{10}: all possible combinations

0.01 sec: Assume it takes 0.01 sec to find f for each combination

8.22 years: total time to find all combinations

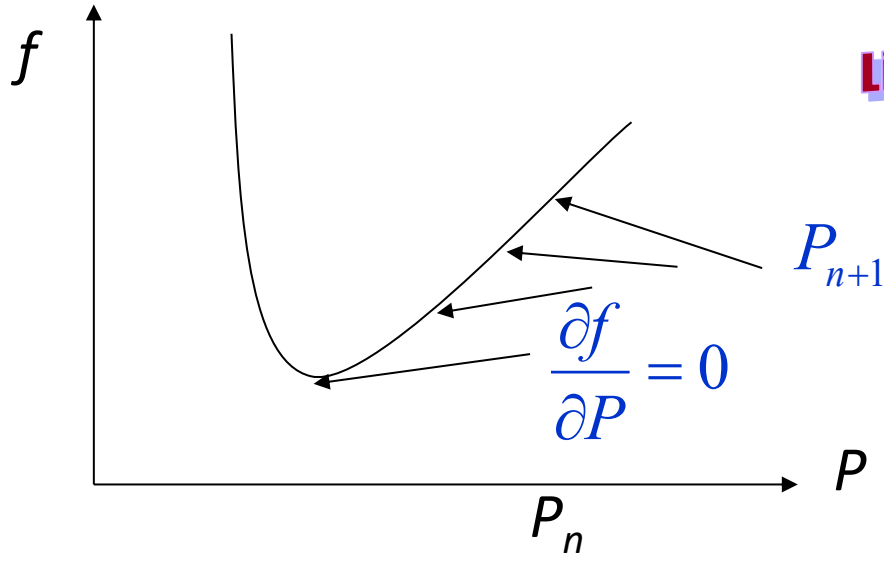
Not practical to evaluate ALL combinations!

Conventional methods – single point search

- Optimization problem: Finding a P_o where

$$P_o = \left\{ P_o \in S \mid f(P_o) = \min_{\forall P_i \in S} f(P_i) \right\}$$

- Gradient-driven search



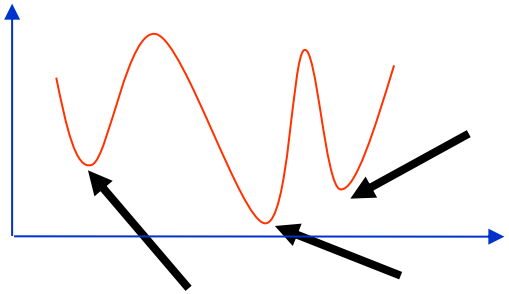
Limitations?

$$P_{n+1} = P_n - \alpha \left. \frac{\partial f}{\partial P} \right|_{P=P_n}$$

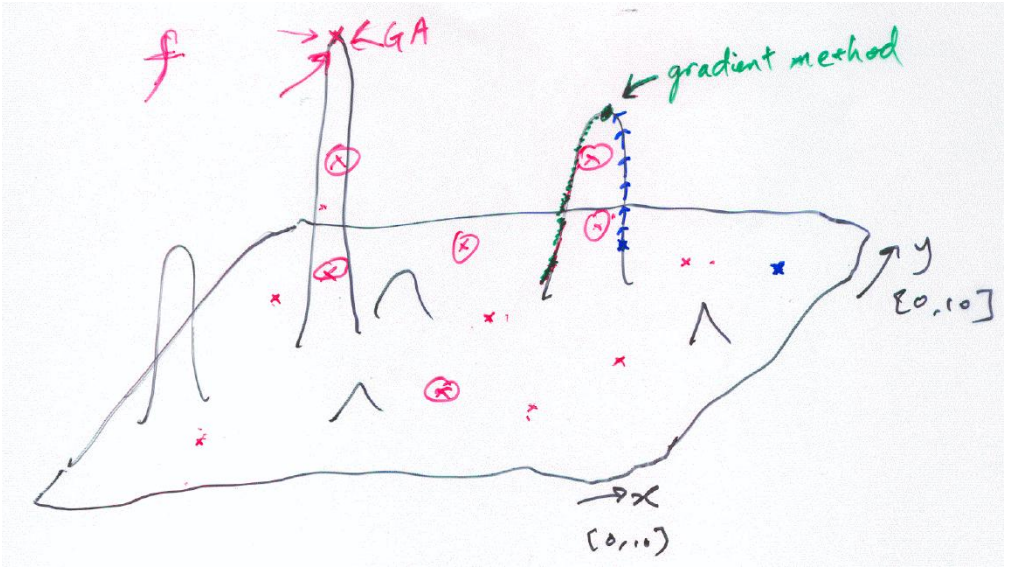
α is a small positive value – learning rate

- **Limitations** 1. Function must be continuous/differentiable

2. Local optima



3. Performance depends on the starting point



Simulated Annealing

- What is simulated annealing? Main idea is to avoid local minima by accepting moves to worse solutions probabilistically
  - One of the first stochastic local search methods
  - Inspired by the analogy to physical annealing
- Physical annealing:
  - A process for obtaining the minimum energy state of a solid, whereby the melted solid is slowly cooled down until reaching its ground state
  - A specific schedule is used to cool down the solid very slowly.
- Computer simulation:
  - Metropolis algorithm is a simulation algorithm for the physical annealing process
- Can be applied to solve optimization problems!

Metropolis Algorithm

- Generates a sequence of states:
  - Current state s_i and next state s_j with energies: E_i, E_j
  - If $E_j \leq E_i$, then accept s_j; otherwise, accept s_j with probability

$$p = \exp(-\frac{E_j - E_i}{k_B T})$$

k_B : Boltzmann constant, T: temperature

- If temperature is lowered slow enough, the solid may reach thermal equilibrium at each temperature.

- What about in the context of optimization?

Simulated Annealing (SA)

- Notations:
  - $\mathcal{S}$: Set of candidate solutions
  - s: Solution in $\mathcal{S}$
  - f: Cost function
  - $f(s)$: Cost function value of solution s
  - $\mathcal{N}(s)$: Neighborhood of s

Without loss of generality, minimization problem is assumed.

1. Generate neighboring solutions s_i from $\mathcal{N}(s)$
2. Accept new solution if it is better in cost value, i.e., when $f(s_i) < f(s)$
3. Otherwise, make the decision whether or not to accept:
   - Yes: Replace current solution with the new (worse) solution
   - No: Discard the new solution
4. Probabilistic decision making based on:
   - Difference between new and old cost values
   - Control parameter *T*

SA: Pseudo-code

Algorithm SIMULATED-ANNEALING

Begin

s_0 = Initial state / solution

T_0 = Initial temperature [positive value]

while (*Not yet frozen*) **do**

while (*Not yet at equilibrium at this temperature*) **do**

Choose s' from the neighborhood of s, $\mathcal{N}(s)$

$\Delta F = f(s') - f(s)$

if $(\Delta F < 0)$ **then (downhill move)**

$s = s'$

else (probabilistic uphill move)

with probability $p = exp\left(-\frac{\Delta F}{T}\right), s = s'$

Random(0,1) < $e^{(-x)}$

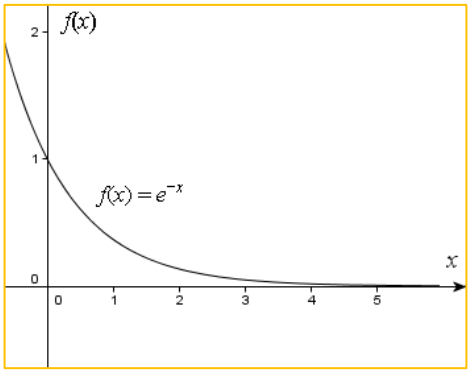
end if

[Might wish to update best-seen solution.]

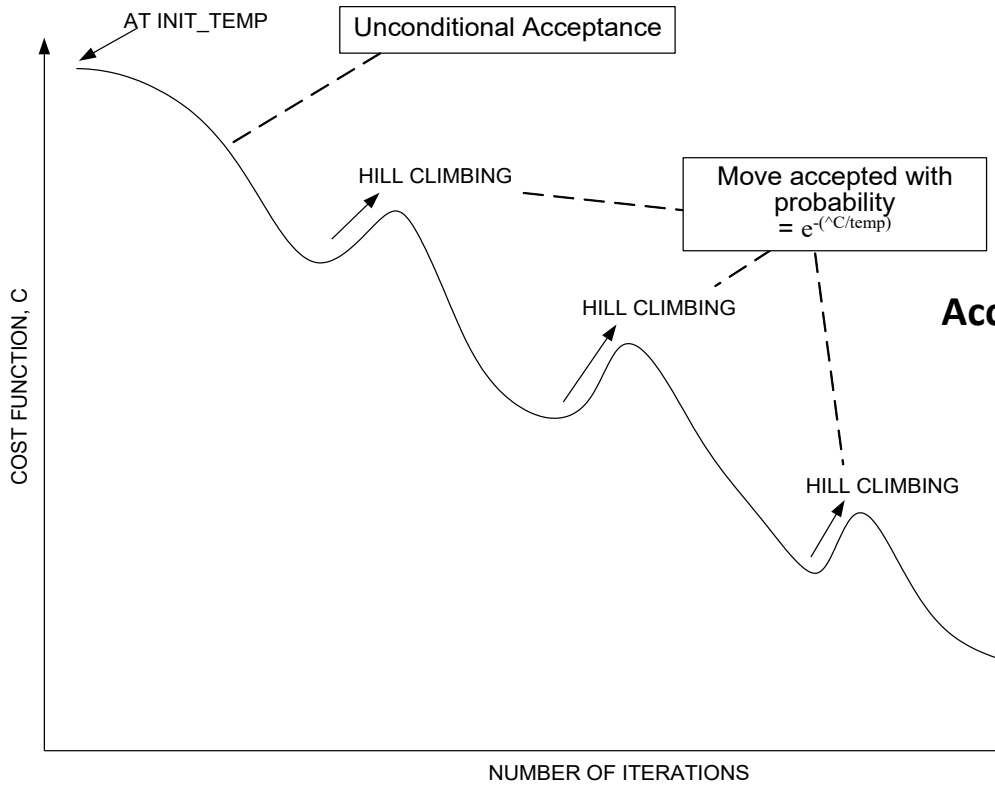
End.

Update temperature T

End.



Simple illustration of SA



Generation of neighborhood solution:

- Generate a random neighboring solution
- Systematically generate neighboring solution, ensuring that neighborhood space is totally explored.

Acceptance criterion:

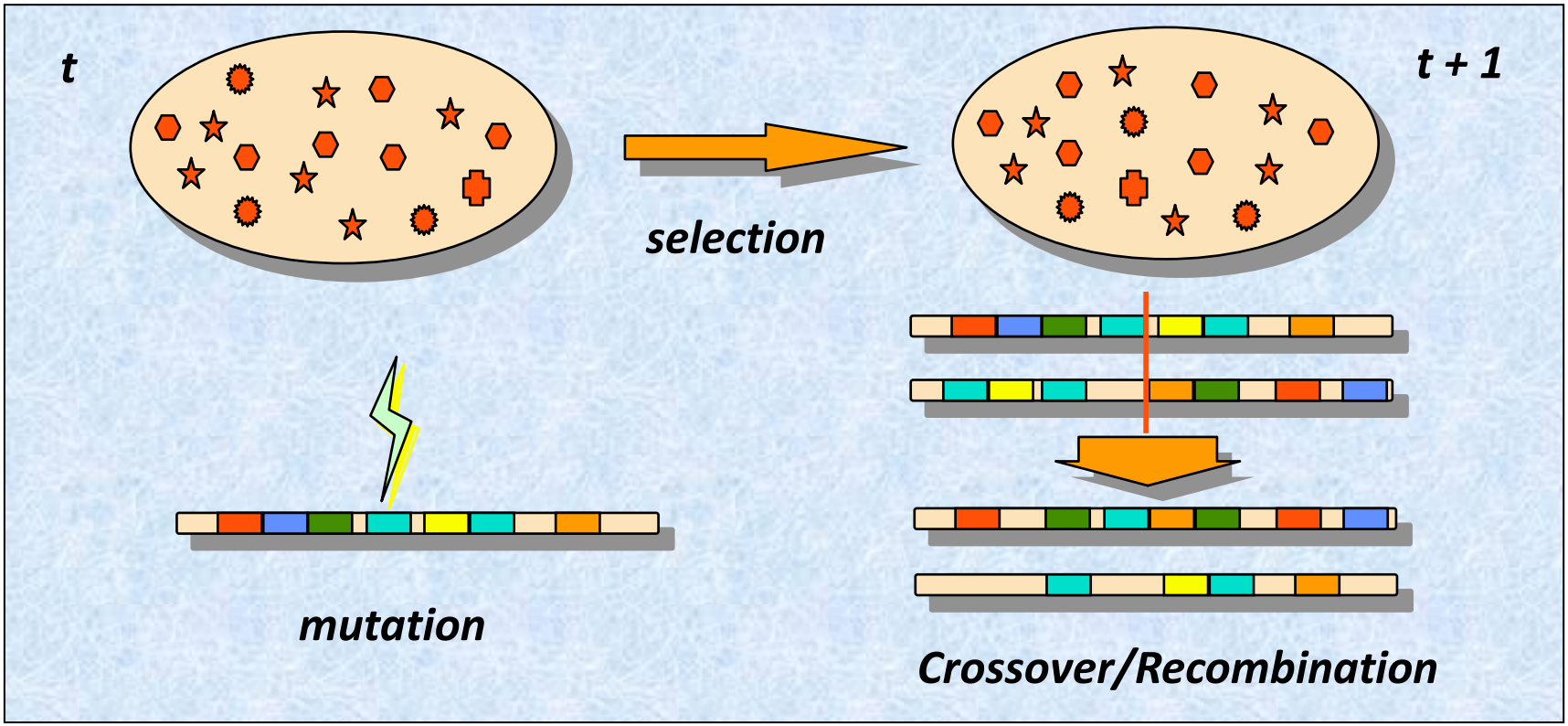
- Metropolis criterion is often used:
- If $f(s') \leq f(s)$ then accept s'
- Otherwise, accept it with probability

$$p = \exp(-\frac{f(s') - f(s)}{T})$$

Genetic Algorithms

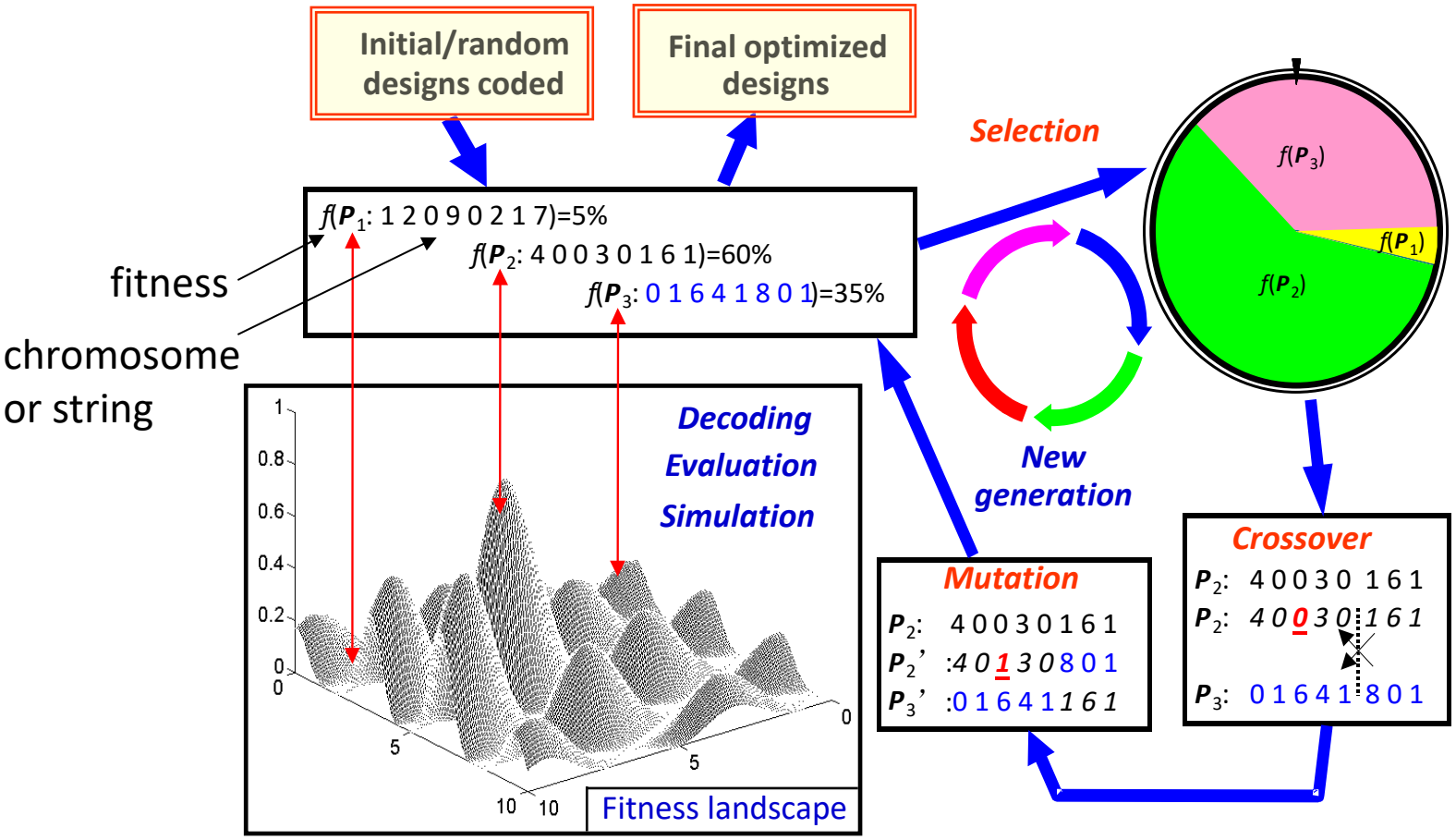
- A genetic algorithm is a search-based algorithm based on natural selection, which can be used for solving AI optimization problems.
- Some basic terminologies:
  - **Population:** This is a subset of all the probable solutions that can solve the given problem.
  - **Chromosomes:** A chromosome/string is one of the solutions in the population.
  - **Gene:** This is an element in a chromosome.
  - **Fitness function:** This is a function that uses a specific input to produce an improved output. The solution is used as the input while the output is in the form of solution suitability.
  - **Genetic operators:** Genetic operators such as Selection, Crossover, Selection, are used for changing the genetic composition of chromosomes.

Genetic Algorithm – The Ingredients



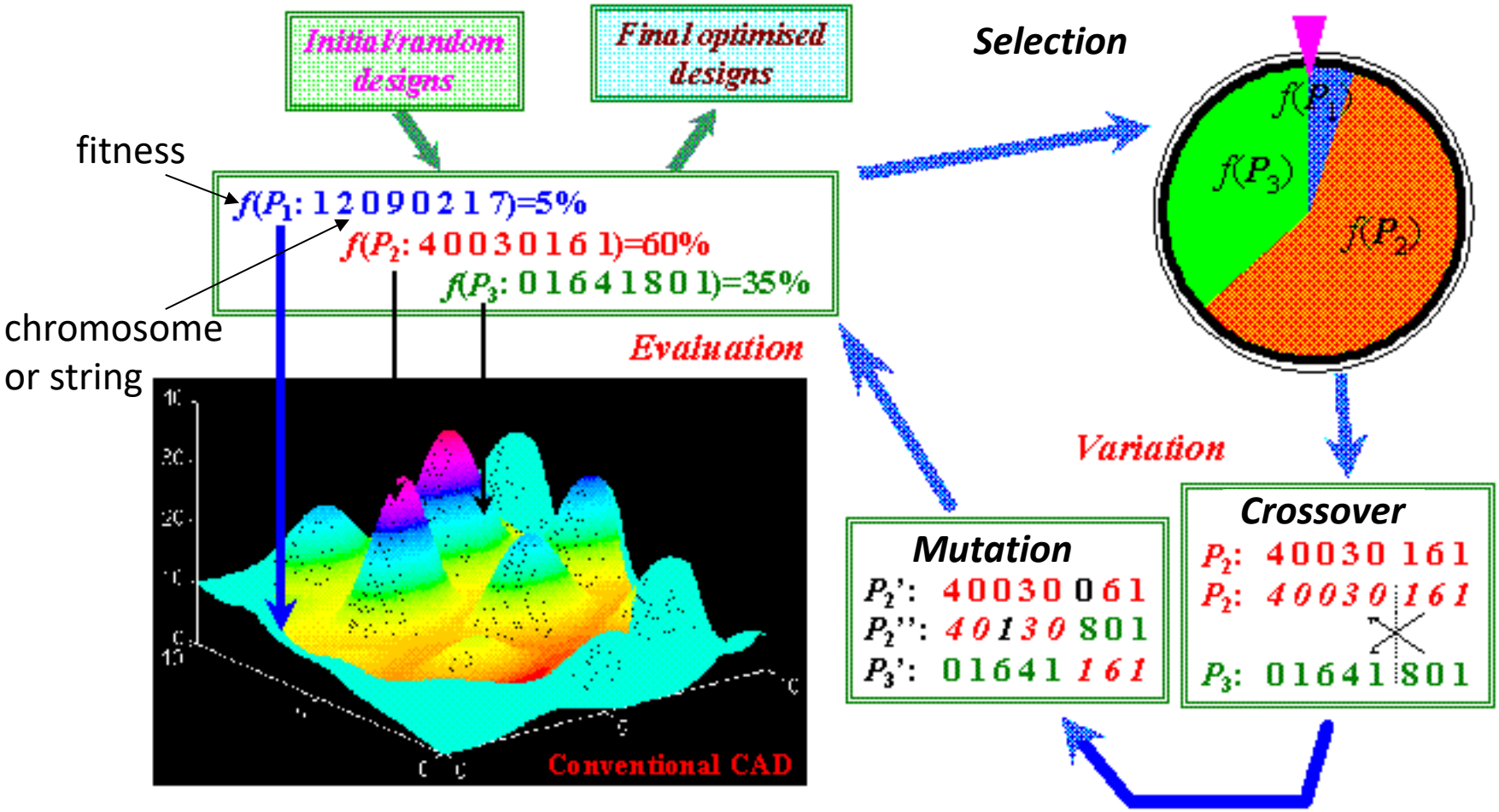
- A genetic algorithm repeatedly modifies a population of individual solutions. At each step, the genetic algorithm selects individuals at random from the current population to be parents and uses them to produce the children for the next generation. Over successive generations, the population "evolves" toward an optimal solution.

A Simple Genetic Algorithm

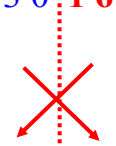


How genetic algorithms work?

$$\text{fitness} \propto \frac{1}{\text{cost}}$$



How genetic algorithms work?

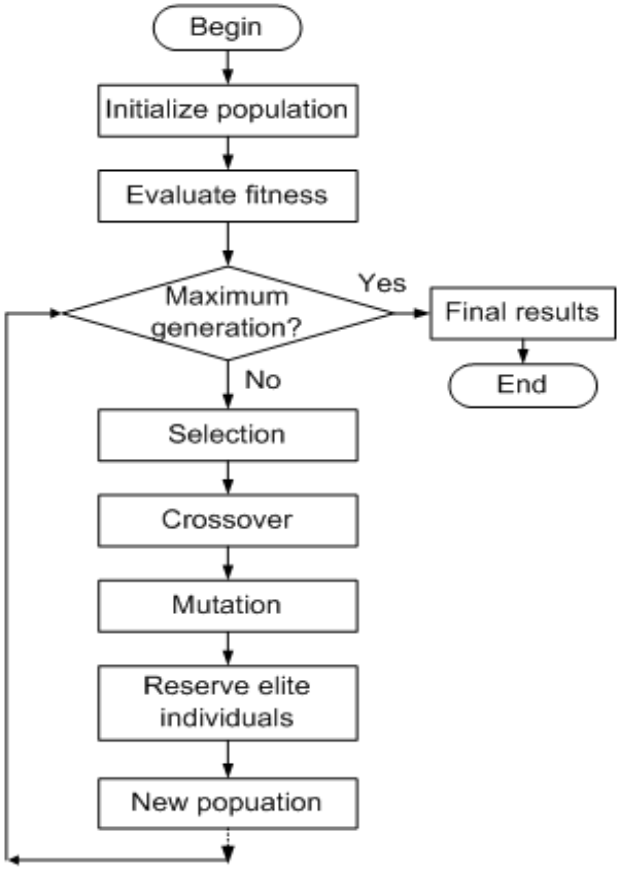
Operations	Chromosomes	Fitness
Initial population: Example of coded parameter sets forming an initial population with size 3. The performance of each parameter set is simulated and then assigned a fitness.	P_1: 1 2 0 9 0 2 1 7 P_2: 4 0 0 3 0 1 6 1 P_3: 0 1 6 4 1 8 0 1	$f(P_1) = 5$ % $f(P_2) = 60$ % $f(P_3) = 35$ % (*N.B. The above fitness values are examples*)
Reproduction: A simple scheme is to allow the chromosomes to reproduce off-spring according to their respective fitness. Thus P_1 has low probability of producing children, P_2 has a probability of producing two and P_3 one.	P_2: 4 0 0 3 0 1 6 1 P_2: 4 0 0 3 0 **1 6 1**  P_3: 0 1 6 4 1 *8 0 1*	Evolution in progress (*No need to re-calculate fitness here*).
Crossover: Some portion of a pair of chromosomes is exchanged at the dotted position randomly specified.	P_2: 4 0 0 3 0 1 6 1 P_2': 4 0 0 3 0 *8 0 1* P_3': 0 1 6 4 1 **1 6 1**	No fitness calculations needed here.
Mutation: The binary values of some genes of some chromosomes are inverted. The value which has been changed as an example is highlighted by an underline.	P_2: 4 0 0 3 0 1 6 1 P_2'': 4 0 **1** 3 0 8 0 1 P_3': 0 1 6 4 1 1 6 1	A new generation is now formed and the fitness needs to be evaluated for the next cycle.

Basic Operations of a Simple GA

Make initial population;
Evaluate the fitness of all initial chromosomes;
REPEAT

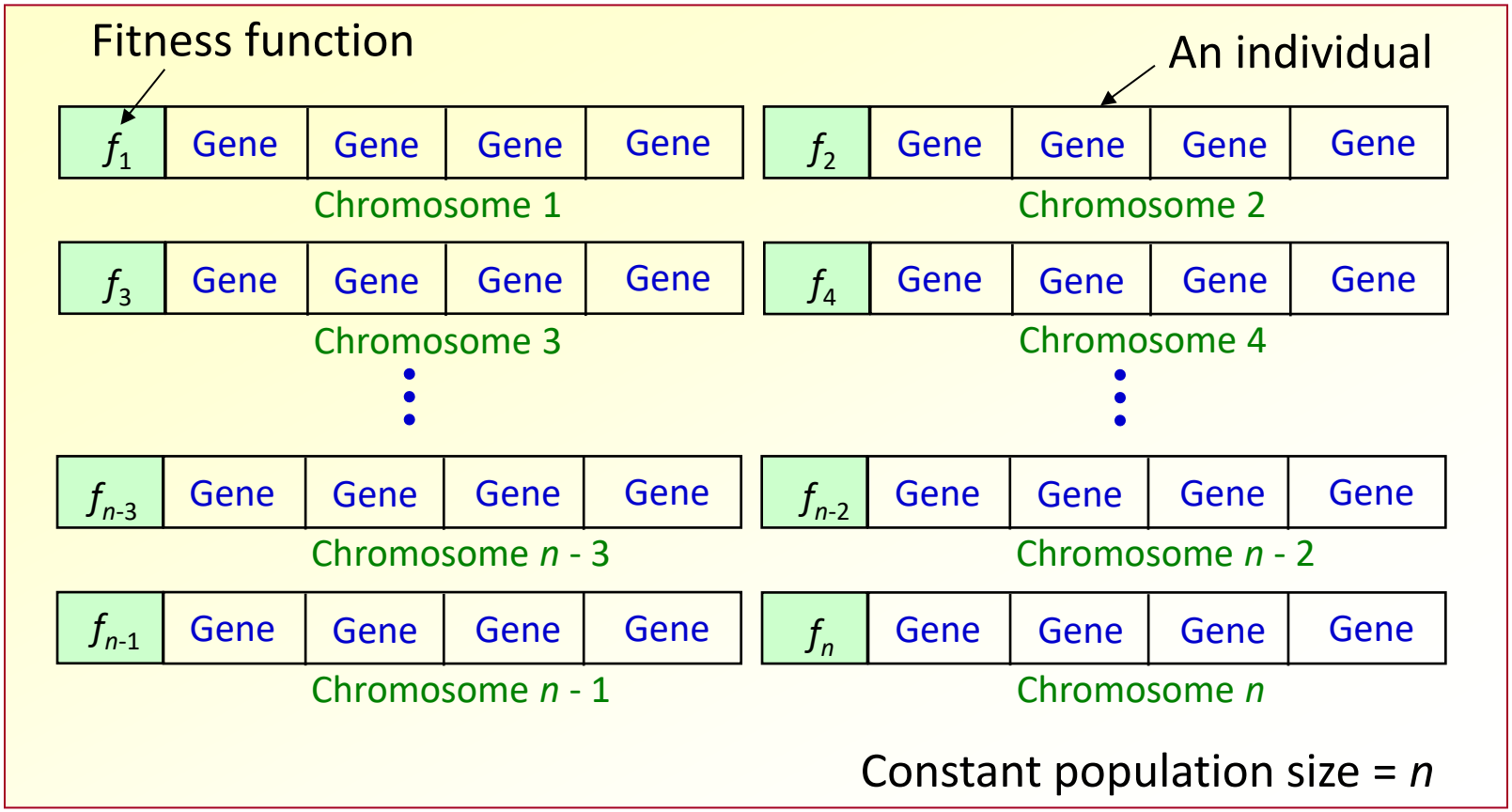
Reproduce children from parents (selection);
Apply crossover to some chromosomes;
Apply mutation to some chromosomes;
Preserve some elite individuals;

UNTIL satisfactory result found
Decode the best chromosome found.



Details of Genetic Algorithms

Structure of population in a GA

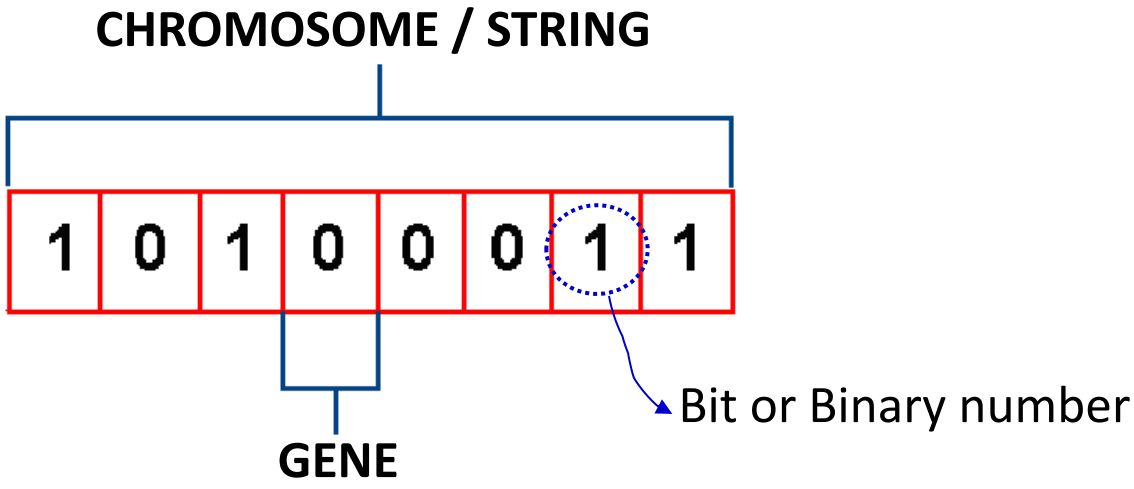


Designing a Representation

- We have to come up with a method of representing an individual as a genotype. binary, decimal, real, order-based?
- When choosing a representation, we have to bear in mind how the genotypes will be evaluated and what the genetic operators might be.

numerical or combinatorial?

Example: Binary Representation



What if we have 10 parameters?
How many bits in total?

for 1 parameter

8 bits Genotype



Phenotype

- Integer
- Real number

0-255

$1*2^7 + 0*2^6 + 1*2^5 + 0*2^4 + 0*2^3 + 0*2^2 + 1*2^1 + 1*2^0$

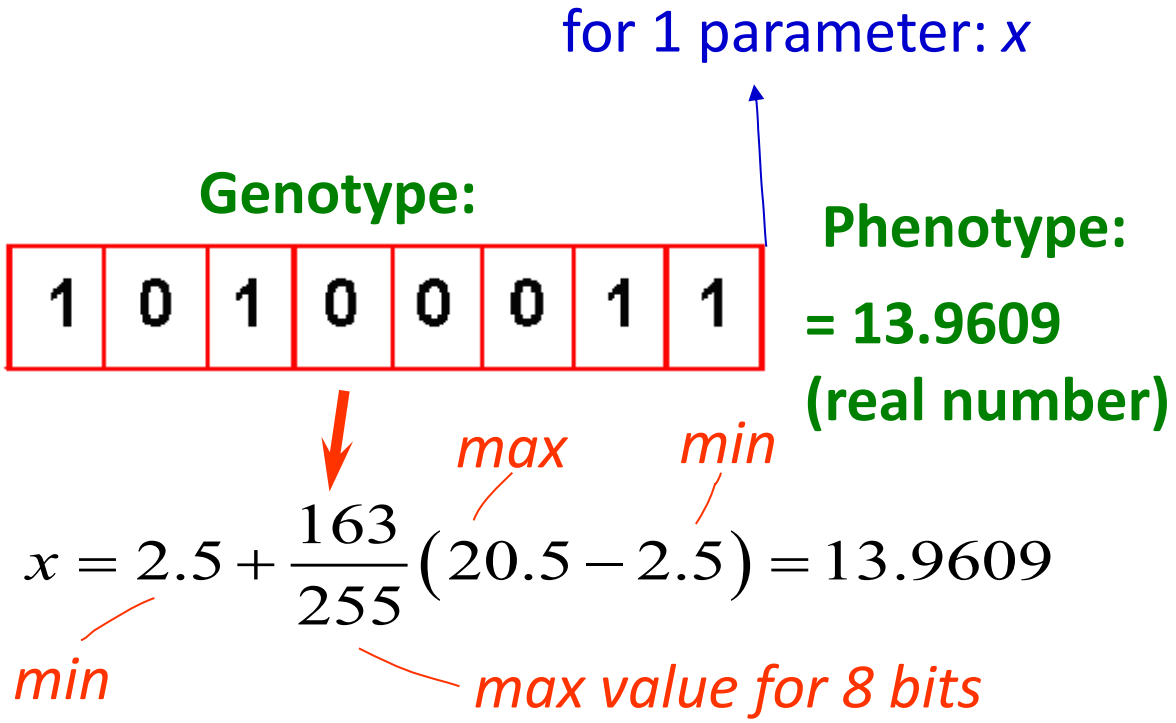
$=128 + 32 + 2 + 1 = 163$

Phenotype:

= 163 (integer)

Could be a real number, e.g., a number between the range of 2.5 to 20.5 using 8 binary bits

min *max*



- What is *x* if the string is: 00000000 ?
- What is *x* if the string is: 11111111 ?

Mutation Operators

Some important points are:

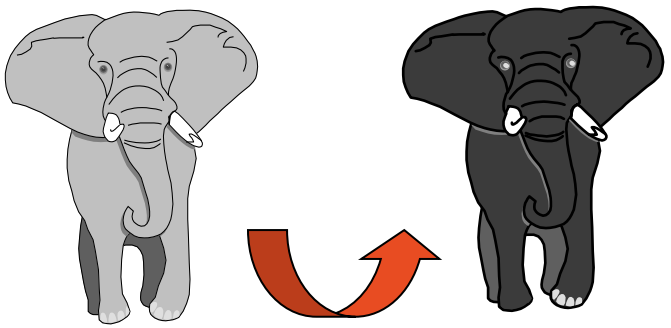
- At least one mutation operator should allow every part of the search space to be reached.
- The size of mutation is important and should be *controllable*.
- Mutation should produce *valid chromosomes*.

before 1 1 1 1 1 1 1

after 1 1 1 0 1 1 1

mutated gene

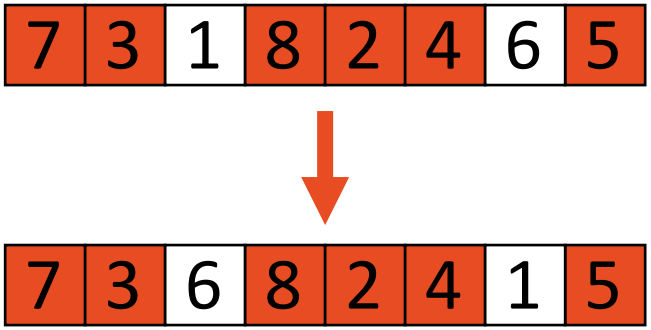
Is mutation critical in GA?



Mutation usually happens with small probability p_m (e.g., 0.02) for each gene.

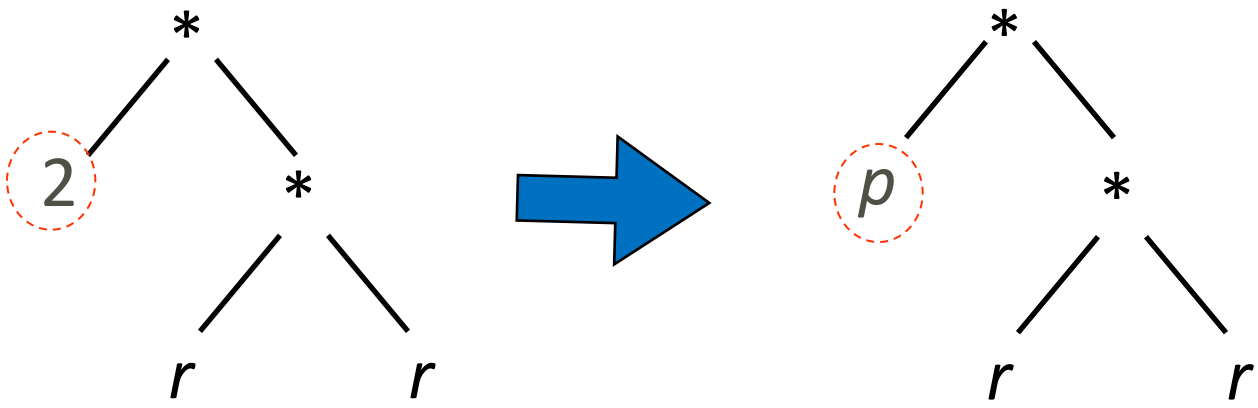
Example: Mutation for order-based representation (Swap)

Randomly select two different genes and swap them.



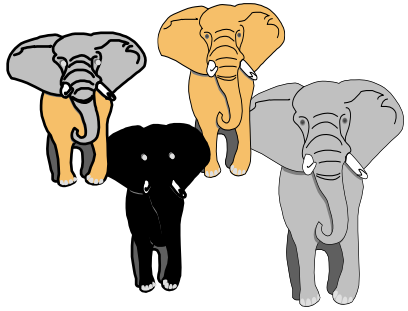
Example: Mutation for tree-based representation

Randomly select one node and replaces it with another random node.

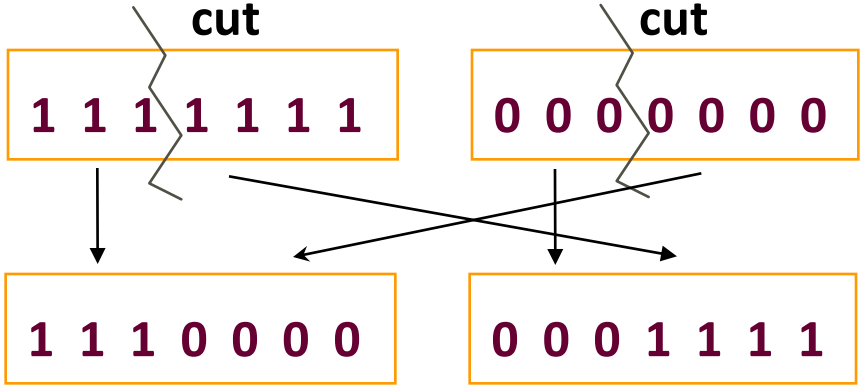


Example: Crossover for Discrete/Binary Representation

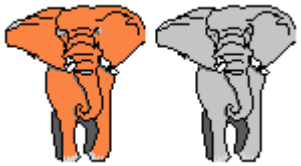
Whole Population:



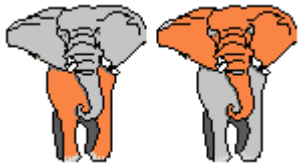
. . .



parents

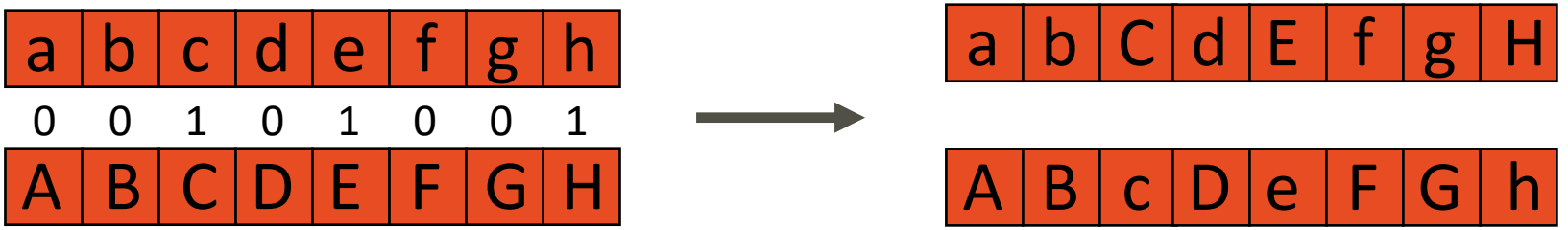


offspring



- The child should inherit something from each parent. If this is not the case then the operator is a mutation operator.
- Crossover should produce valid chromosomes.

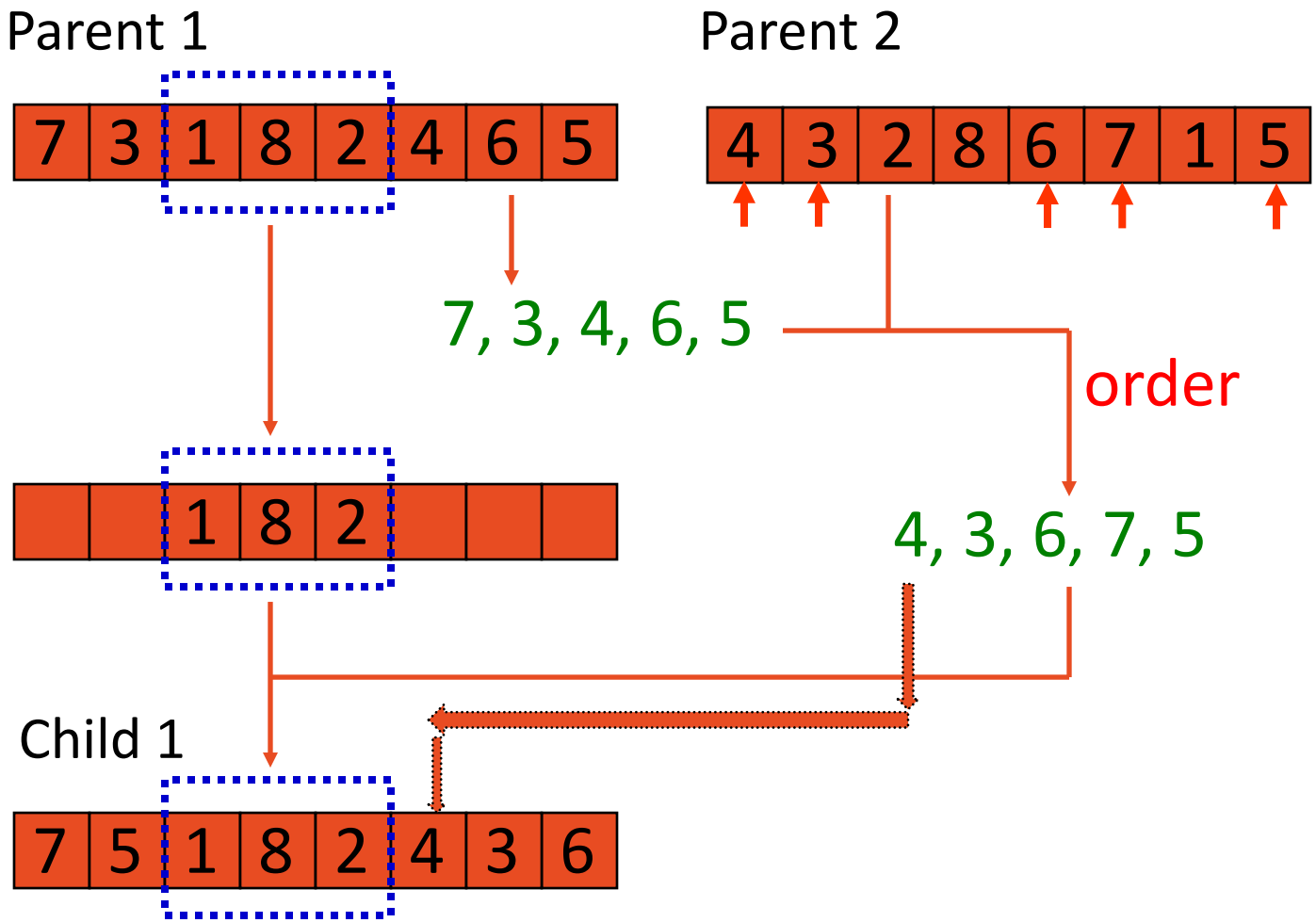
Uniform Crossover



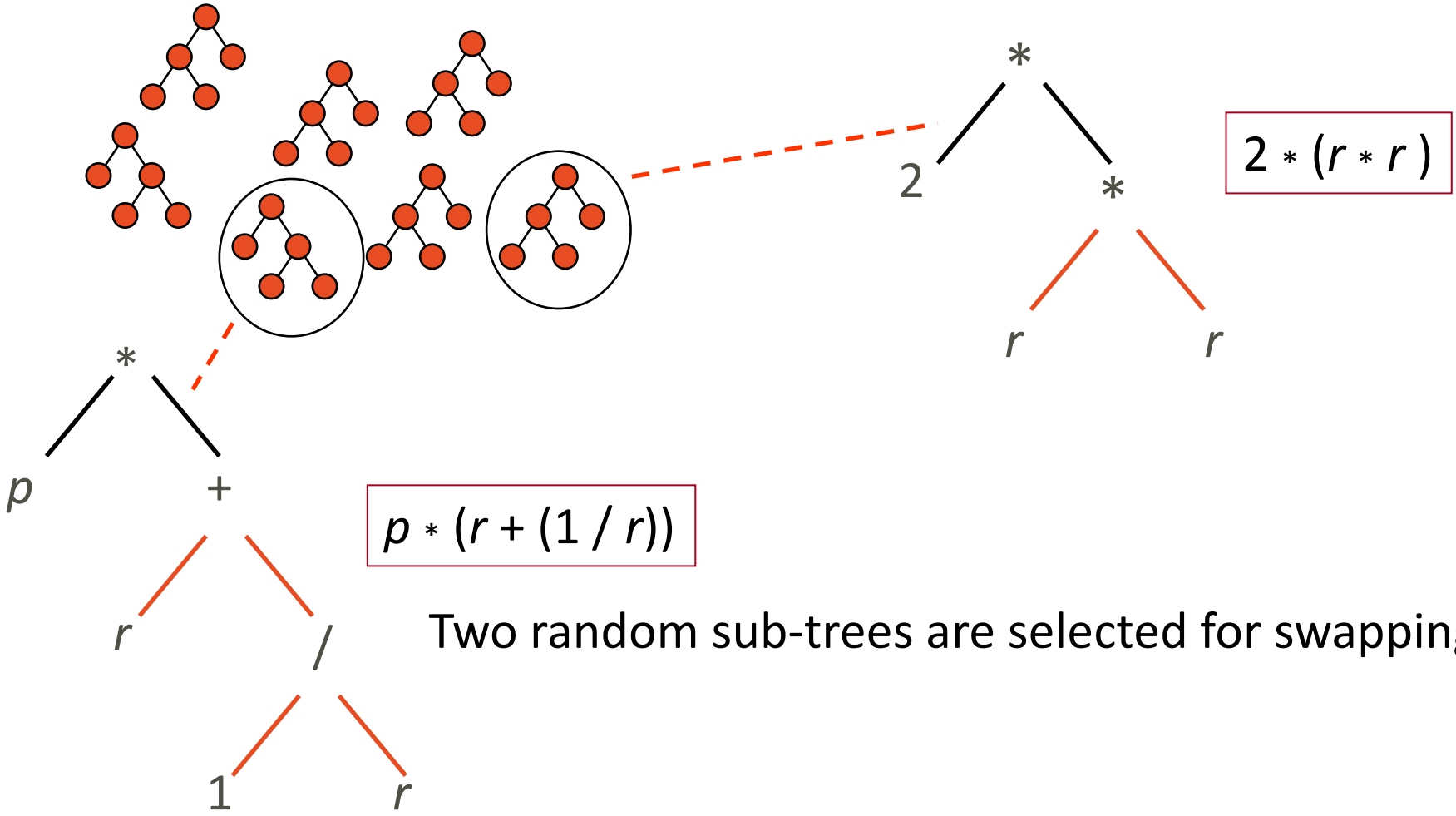
Recombination for order-based representation (*example*):

- Choose an arbitrary part from the first parent and copy this to the first child
- Copy the remaining genes (from the second parent) that are not in the copied part to the first child:
  - Starting right from the cut point of the copied part
  - Using the order of genes from the second parent
  - Wrapping around at the end of the chromosome
- Repeat this process with the parent roles reversed

Example: Recombination for order-based representation



Example: Recombination for tree-based representation



Two random sub-trees are selected for swapping.

Selection Strategy

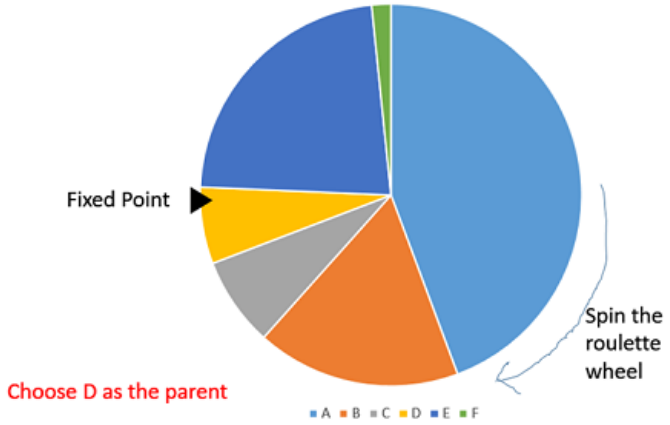
- We want to have some way to ensure that better individuals have a better chance of being parents than less good individuals.
- This will give us selection pressure which will drive the population forward.
- We have to be careful to give less good individuals at least some chance of being parents – since they may include some useful genetic material.
- The most commonly used selection methods include Roulette Wheel Selection, Rank Selection, Tournament Selection etc.

Example: Fitness proportionate selection

Probability of x_i being selected for mating is: $f_i / \bar{f}$

f_i = fitness of x_i $\bar{f}$ = total fitness in the population

- Better (fitter) individuals have:
  - more space
  - more chances to be selected



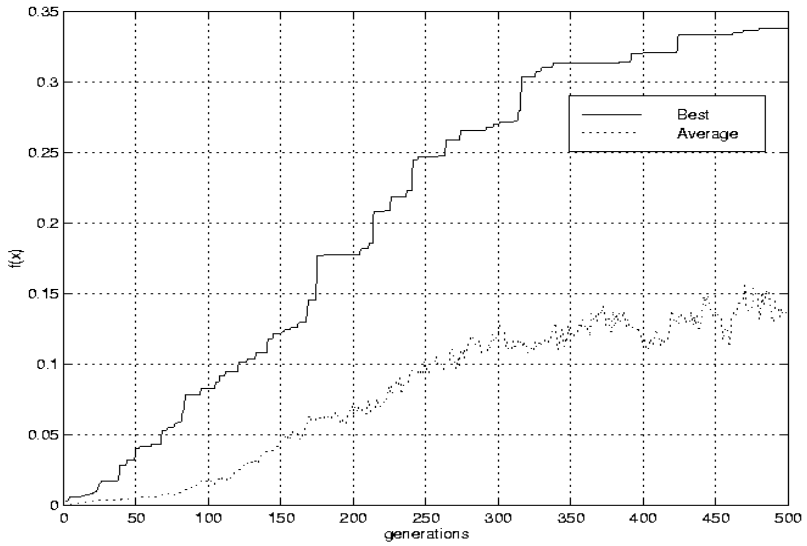
Chromosome	Fitness Value
A	8.2
B	3.2
C	1.4
D	1.2
E	4.2
F	0.3

Disadvantages:

- Danger of premature convergence because outstanding individuals take over the entire population very quickly.
- Low selection pressure when fitness values are near each other.

Elitism in GA

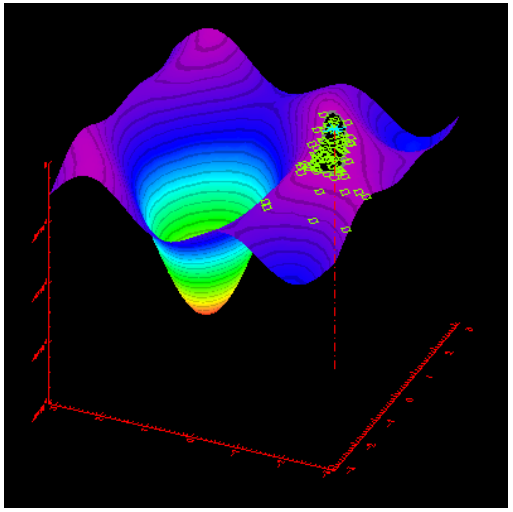
- Should fitness be constantly improved?
  - Re-introduce in the population previous best-so-far (elitism).
  - For example, keep the best 10% of parent population and re-introduce them in the next generation by replacing the worst 10% children population.
  - In GA, elitism tends to improve on the results.



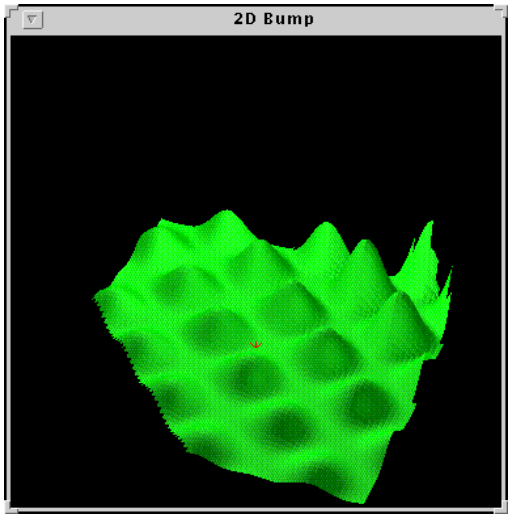
Example of a convergence trace

Stopping Criterion

- The optimum is reached!
- Limit on CPU resources: maximum number of fitness evaluations.
- Limit on the patience: after some generations without improvement.
- Never draw any conclusion from a single run
  - Use statistical measures (e.g., average, standard deviation).
  - Obtain results from a number of independent runs.



Moving Optimum



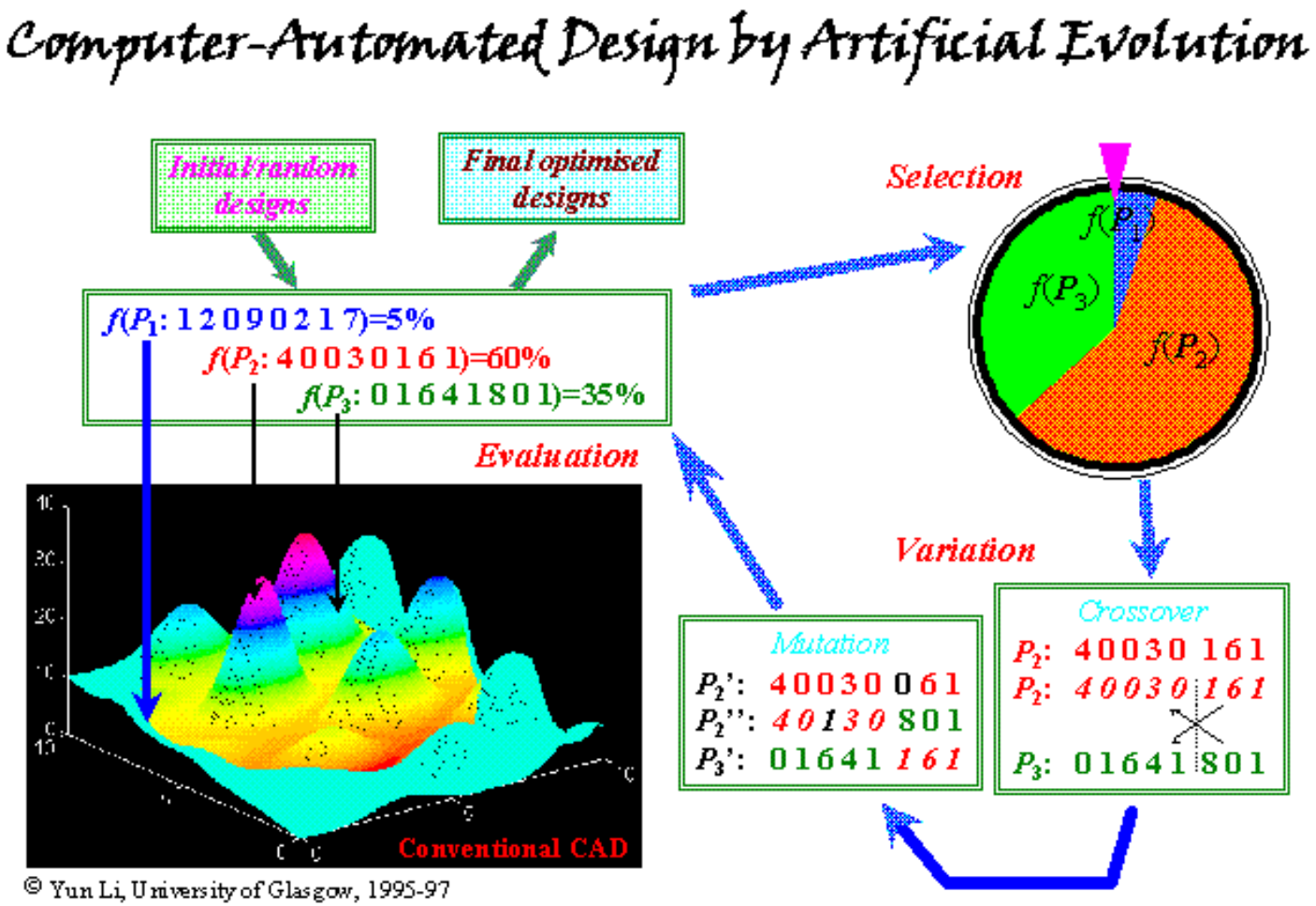
Comparison

Advantages of genetic algorithms

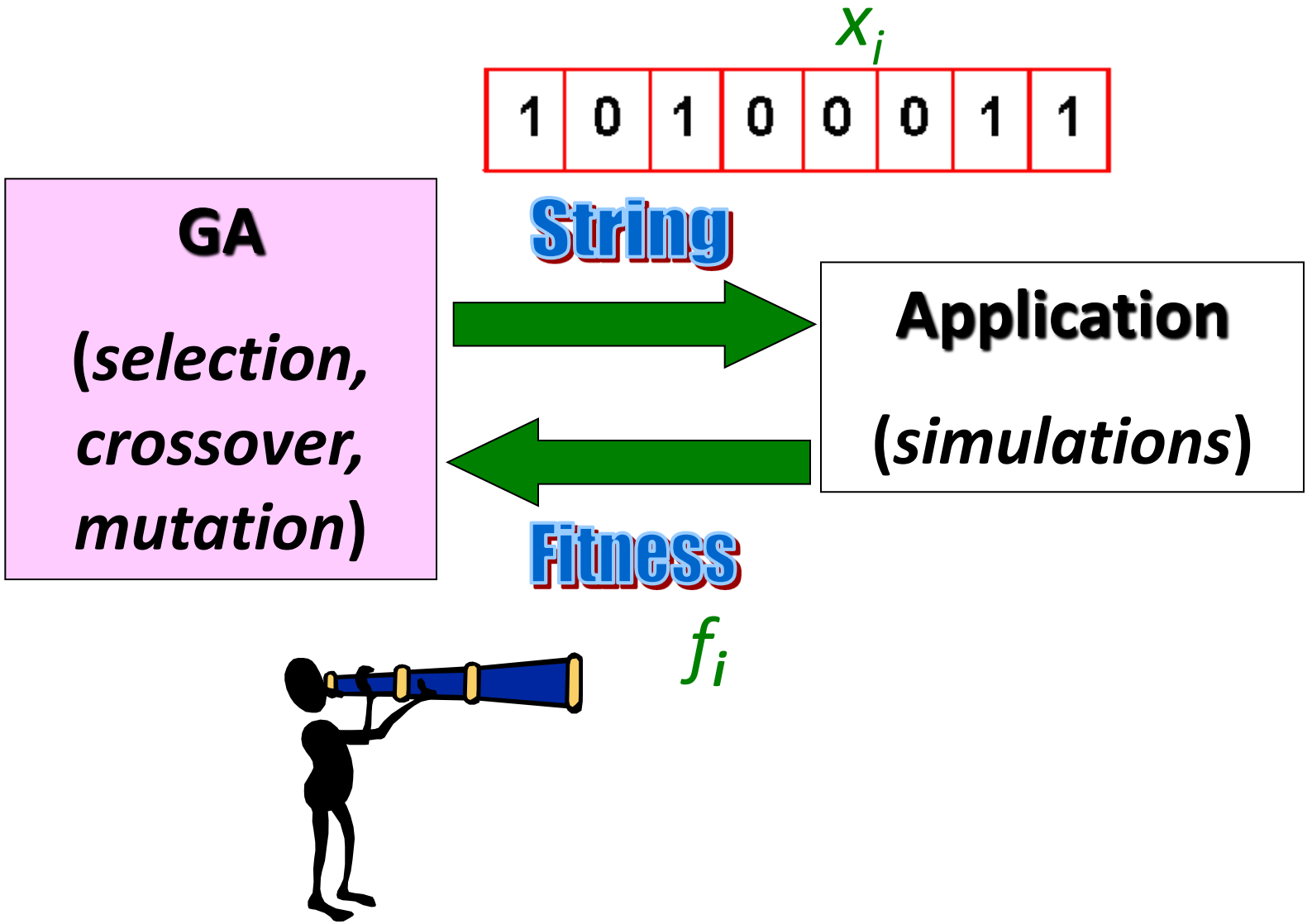
- It has excellent parallel capabilities.
- It can optimize various problems such as discrete functions, multi-objective problems, and continuous functions.
- It provides answers that improve over time.
- A genetic algorithm does not need derivative information.

Limitations of genetic algorithms

- They are not effective in solving simple problems.
- Lack of proper implementation may make the algorithm converge to a solution that is not optimal.
- The quality of the final solution is not guaranteed.
- Repetitive calculation of fitness values may make some problems to experience computational challenges.



Question: Why such an evolution process will lead to better solutions?


x_i
1 0 1 0 0 0 1 1
String
GA
(selection, crossover, mutation)
Application
(simulations)
Fitness
f_i

A GA's View of a Population

String	*Objective function value*
10111	10
01000	5
11010	3
00011	20

Task: Maximize objective function value

Question: Can you guess what is a better string might be?

Schemata Theorem

- A *schema* is a set of strings that share certain values
- Example: **1** matches strings {11101, 11100, 01101,…}

Schema	**1**	*1**1	1*10*
11101	✓	✓	✓
11100	✓		✓
01101	✓	✓	

The Schemata Theorem – The processing power of GA is understood in more rigorous terms by examining the growth rates of the various schemata or similarity templates contained in a population.

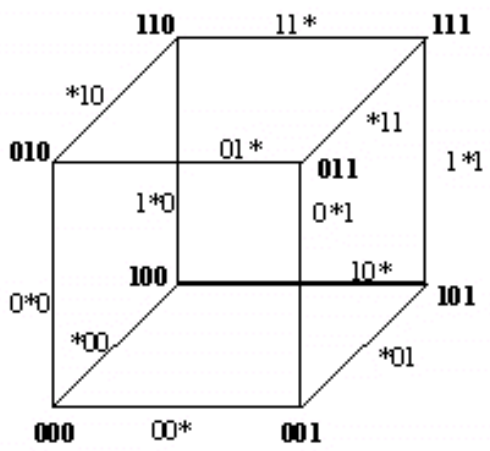
A schema matches a string if at every location in the schema a "1" matches a "1", a "0" matches a "0" or a "*" matches either.

For cardinality "k", there are $(k+1)^l$ schemata in a string; l is the length of the string, eg., k =2 (*binary coding*), l = 4 $\rightarrow$ 81 schemata.

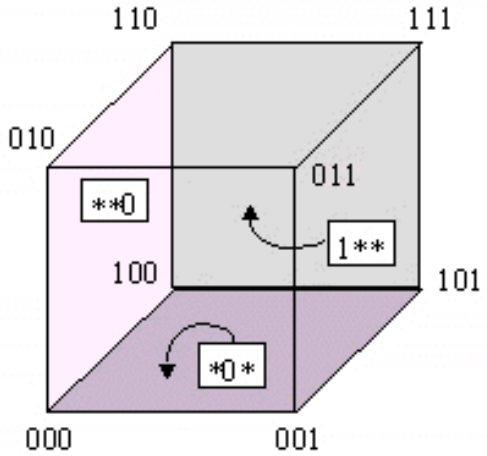
Order of a schema, $o(S)$ is defined as the number of fixed positions.

The defining length, $d(S)$ of a schema is the distance between the first and the last specific fixed position.

E.g., S_1 = *1001*, S_2 = 1****0, then $O(S_1) = 4$, $O(S_2) = 2$, $d(S_1) = 3$, $d(S_2) = 5$



Search space for a problem with L=3 and K=2 (binary alphabet)



Three of the six schemata of order 1

Order of the Schema o(H)	Hyperplane Dimension	Geometric Interpretation	Strings in the Schema	Number of Schemata
3	0	Point	1	8
2	1	Line	2	12
1	2	Plane	4	6
0	3	Cube	8	1
			Total	**27**

$(k+1)^l = (2+1)^3 = 27$

Let us consider the individual and combined effect of genetic operators:

Effect of Selection

Fitness of a schema *fit*(*S*) = averaging the fitness values of all strings in the population that match this schema

$$fit(S) = \frac{\sum_{k=1}^{r} fit(x_k)}{r}$$

$r = n_s(t)$ is the number of strings matching S in the current generation

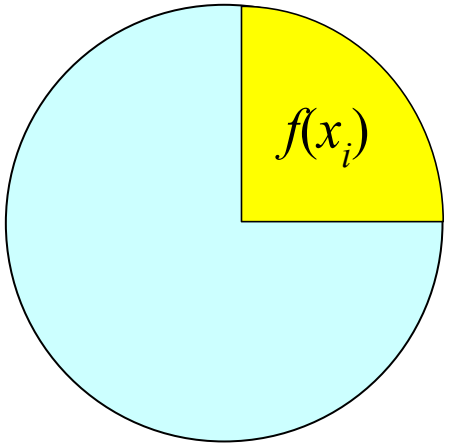
Considering the i-th string x_i is selected with the probability

$$P_i = \frac{fit(x_i)}{\sum_{j=1}^{N} fit(x_j)}$$

N is the total number of strings in the population

- Probability of x_i to be selected:

$$P_i = \frac{\text{fitness of } x_i}{\text{total fitness}} = \frac{fit(x_i)}{\sum_{j=1}^{N} fit(x_j)}$$



- Number of x_i to be selected for a population size of N:

$$Nr(x_i) = N \cdot P_i = N \cdot \frac{fit(x_i)}{\sum_{j=1}^{N} fit(x_j)}$$

The number of strings matching *S* in the next population equals

total fitness of strings matching S

total fitness of all strings in the pop.

$$n_s(t+1) = N \cdot \frac{\sum_{k=1}^{r} fit(x_k)}{\sum_{j=1}^{N} fit(x_j)} = \frac{r\,\frac{\sum_{k=1}^{r} fit(x_k)}{r}}{\frac{\sum_{j=1}^{N} fit(x_j)}{N}} = r\frac{fit(S)}{< fit >} = n_s(t) \cdot \frac{fit(S)}{< fit >}$$

A particular schema grows as the ratio of the *average fitness of the schema* to the *average fitness of the population* → Schemata with fitness values above the population average will receive an increasing number of samples in the next generation and vice versa.

Assume that a particular schema remains ***above average*** with an amount ε<*fit*>, then

Grows up exponentially

$$n_s(t+1) = n_s(t) \cdot \frac{< fit > +\varepsilon < fit >}{< fit >} = (1+\varepsilon) n_s(t)$$

Effect of Crossover

Assuming that the crossover point is uniformly distributed across the string, the *probability of destruction* of *S* depends upon the defining length of the schema = $\frac{d(S)}{l-1}$

Examples:

H_1 = * 1 * * * * 0 $\frac{5}{6}$

H_2 = * * * 1 0 * * $\frac{1}{6}$

The *probability of survival* of *S* is $1-\frac{d(S)}{l-1}$

Crossover occurs at probability of P_{cross}, so the probability of survival of *S* is higher and equal to (*l* = length of the string):

$$1-P_{cross}\frac{d(S)}{l-1}$$

So, the combined effect of *reproduction and crossover* is:

$$n_s(t+1) = n_s(t) \cdot \frac{fit(S)}{< fit >}\left(1 - P_{cross}\,\frac{d(S)}{l-1}\right)$$

Effect of Mutation

With the mutation probability P_m, the survival of schema is equal to

$$P_{survival} = (1 - P_m)^{o(S)}$$

Example:

H = * 1 * * 0 * *

$(1- P_m)(1- P_m)$

Assuming P_m << 1, the following approximation holds

$$(1 - P_m)^{o(S)} = 1 - P_m o(S)$$

Combined effect of selection, crossover and mutation:

$$n_s(t+1) = n_s(t) \cdot \frac{fit(S)}{< fit >}\left(1 - P_{cross}\frac{d(S)}{l-1}\right)\left(1 - P_m o(S)\right)$$

$$n_s(t+1) \approx n_s(t) \cdot \frac{fit(S)}{< fit >}\left(1 - P_{cross}\frac{d(S)}{l-1} - P_m o(S)\right)$$

Schema Theorem - *Short defining length*, *low order* and *above-average* schemata receive exponentially increasing trials in subsequent generations.

Low order, short defining length, highly fit schemata are called the *building blocks* of the search. When the building blocks are pieced together, similar to solving the "Jigsaw Puzzle", the solution emerges.